

Lecture 9 — Behaviour-Driven Development & Verification

Lecture id: L09 **Source decks:** BDD (~30p) **Labs:** BDDLab (~1p) **Process phase(s):** Verification
Citation key: (BDD p.X) ; readings [Raj13] etc. **Grounding note:** Every non-obvious claim below is cited to a specific slide of BDD.pdf as (BDD p.X) or to the lab handout as (BDDLab p.1) . The deck (titled "Software Verification — Pragmatic BDD for Java", by Jan Corfixen Sørensen, USD) is **tool-focused**: it teaches BDD as a *practice for the Verification phase* and then drills into two concrete Java libraries — **JGiven** (a developer-friendly BDD framework) and **AssertJ** (a fluent assertion library, including AssertJ-Swing for GUI tests). The deck does **not** mention JHotDraw by name and does **not** explicitly walk the eight-phase change process; those anchors are supplied here from the course frame [Raj13] and the course overview, and are labelled as such so you can tell deck-grounded fact from course-frame synthesis. Reading keys: [Raj13] = Rajlich textbook; [Nor06] = Dan North, *Introducing BDD*; [MC09] = Martin & Coplien, *Clean Code*; [Fow11] / [Fowler99] = Fowler refactoring; [GHJV94] = Gang-of-Four design patterns.

Overview

This lecture sits at the **end of the software-change process** — the **Verification** phase. After a change has been initiated, located (Concept Location), analysed (Impact Analysis), prepared (Prefactoring), implemented (Actualization), cleaned up (Postfactoring) and concluded [Raj13], Verification asks the closing question: *is the change correct, complete, and free of regressions?* The deck answers that question with **Behaviour-Driven Development (BDD)** — a style of writing automated, executable acceptance criteria in language that domain experts (not just developers) can read, so the tests double as **living documentation** (BDD p.4).

Why this matters for maintenance specifically: a maintained system is changed over and over, so its safety net of tests is read, edited, and re-run far more often than it was first written. Tests that are cryptic, duplicated, or developer-only rot quickly and stop being run — and an un-run test verifies nothing. BDD's whole proposition is to make the verification artefacts *cheap to read and cheap to keep* by phrasing them as behaviour in the domain's vocabulary, which is exactly the property the Verification phase needs from a maintenance lifecycle.

The deck has three movements:

1. **Why BDD / what BDD is** (BDD p.3–5) — motivates BDD by the failings of ordinary unit tests and introduces the **Given / When / Then** scenario format with a running "pancake" example.
2. **BDD frameworks, focusing on JGiven** (BDD p.6–20) — surveys "classical" plain-text BDD frameworks (Cucumber, JBehave, ...) and their maintenance cost, then "developer-friendly" frameworks, then spends the bulk of the deck on **JGiven**: scenarios as plain Java methods, **stage classes**, **state transfer between stages** via annotations, a full worked pancake example, console + HTML5 reports, and an industrial experience report from TNG.
3. **AssertJ** (BDD p.21–30) — a fluent assertion library used *inside* the Then steps: motivation (JUnit/Hamcrest/Fest pain), basic fluent use, custom `Condition`s, custom assertions, and **AssertJ-Swing** for driving GUI tests.

The accompanying lab (BDDLab) closes the loop to requirements: it shows how to map **user stories** ("As a ... I want ... so that ...") onto **Given-When-Then** scenarios and asks you to automate them with **JGiven** + **AssertJ** (+ **AssertJ-Swing** for Swing apps) (BDDLab p.1). The connective tissue for the exam: **BDD scenarios are the acceptance criteria that the Verification phase checks**, and JGiven/AssertJ are the concrete Java tooling that makes those criteria *executable*.

The whole deck is 30 slides; this guide covers every one of them (see the Source Map at the end).

Deck anatomy at a glance

Structurally, the 30 slides divide cleanly: **three divider slides** mark the parts ("Behavior-Driven Development" p.2, "JGiven" p.8, "AssertJ" p.21); **five code slides** carry the JGiven example (the scenario method p.10 and the three stages p.14–16) plus two AssertJ examples (p.25, p.27); **two screenshot slides** show the reports (console p.17, HTML5 app p.18); **one diagram slide** shows state transfer (p.12); and the rest are bullet slides. The deck's whole argument can be read off this anatomy: a *problem* (p.3), a *practice* that solves it (p.4–5), a *framework survey* (p.6–7), a *deep tool dive with one running example* (p.9–18), *industrial evidence and an honest trade-off* (p.19–20), and a *second tool* for the assertion layer with the same motivate-demonstrate-extend rhythm (p.22–29), closing on documentation pointers (p.30) (BDD p.1–30).

For revision priorities: the highest-yield single slides are **p.4** (BDD's definition), **p.13** (the three state annotations), **p.20** (JGiven's strengths and its one limitation), **p.28** (the custom-assertion recipe), and **p.29** (AssertJ-Swing's capabilities); the highest-yield code to be able to reproduce from memory is the p.10 scenario method and the skeleton of the three stages (p.14–16). The lab page (BDDLab p.1) supplies the requirements-side vocabulary (user stories, both templates, the three Figure-1 mappings) that the deck assumes.

Learning Objectives

After this lecture you should be able to:

1. **Explain why BDD exists** — articulate the typical problems with ordinary tests (too many irrelevant technical details, the point of the test is hard to grasp, code duplication, readable only by developers, unusable as documentation) and how BDD addresses them (BDD p.3–4).
2. **Define BDD** as behaviour described in a **common domain language** understandable by domain experts, with experts and developers **collaborating** to define behaviour, **executed like normal tests**, producing **living documentation** (BDD p.4).
3. **Read and write Given/When/Then (Gherkin-style) scenarios**, e.g. the pancake scenario (BDD p.5).
4. **Distinguish "classical" plain-text BDD frameworks** (Cucumber, JBehave, Conordion, Fitnesse, RobotFramework) — which add a **maintenance cost** — from **developer-friendly** frameworks (Spock, ScalaTest, Jnario, Serenity, JGiven) (BDD p.6–7).
5. **Use JGiven**: write a scenario as a Java test method whose calls read like Given/When/Then (BDD p.9–10); build scenarios from **stage classes** that give modularity and reuse (BDD p.11); transfer state between stages with `@ScenarioState`, `@ProvidedScenarioState`, `@ExpectedScenarioState` (BDD p.12–16); and read the **console** and **HTML5** reports JGiven generates (BDD p.17–18).
6. **Critically evaluate JGiven** from the TNG experience report (3 years, up to 70 developers, 3000+ scenarios; readability/reuse up, maintenance cost down; but **domain experts cannot write** JGiven scenarios) (BDD p.19–20).

7. **Use AssertJ** in Then steps: the `assertThat()` factory, fluent chained assertions (`.contains().startsWith()`), good use of polymorphism, custom `Condition` functors (`is / isNot`), and custom type-specific assertions by subclassing `AbstractAssert` and `Assertions` (BDD p.22–28).
8. **Use AssertJ-Swing** to automate GUI verification — simulate user interaction (drag 'n drop), reliable component lookup, embed screenshots of failed GUI tests in HTML reports (BDD p.29).
9. **Map user stories to BDD scenarios** and automate them with JGiven + AssertJ (+ AssertJ-Swing for Swing) for the lab portfolio (BDDLab p.1).
10. **Locate BDD within the change process**: BDD scenarios are the executable **acceptance criteria** of the **Verification** phase [Raj13]; they confirm the actualized change is correct without regressions.
11. **State both user-story templates** from the lab handout — "As a [user type], I want [some goal] so that [some reason]" and the alternative "As a [user type], I want [some goal] because [why]" — and reproduce all three Figure-1 story→scenario mappings (calculator, Math teacher, QA engineer) (BDDLab p.1).
12. **Read a JGiven HTML5 report screen**: locate the summary counts (All / Failed / Pending scenarios), the tag and class navigation in the sidebar, the per-scenario tag chips and durations, and the Group By / Sort By / Status / Tags / Classes controls (BDD p.18).
13. **Narrate the line-by-line mechanics of the stage classes** — the `Stage<SELF>` self-type, `snake_case` step names becoming report prose, intra-stage step delegation (`an_egg()` → `the_ingredient("egg")`; `the_resulting_meal_is_a_pancake()` → `the_resulting_meal_is_a("pancake")`), and why Then steps may be `void` (BDD p.14–16).

Key Concepts

What BDD is (and why) — behaviour in a shared language

What it is. Behaviour-Driven Development is a way of specifying and verifying software in terms of its *externally observable behaviour*, written in a **common domain language understandable by domain experts**. The deck's definition has four interlocking parts (BDD p.4): (1) behaviour is described in domain language so non-developers can read it; (2) domain experts and developers **collaborate** to define that behaviour; (3) the description is then **executed like normal tests**; and (4) the running tests form a **living documentation** of the system. It is not a tool and not a test framework — it is a *practice*, of which JGiven and AssertJ are concrete implementations.

What it's used for / why it matters. BDD exists to fix the recurring failures of conventional automated tests. The deck names the "typical test issues" that motivate it (BDD p.3):

- **Many technical and often irrelevant details** clutter the test, so the behaviour is buried under setup, mocks, and plumbing.
- **The point of the test is often hard to grasp** — you can read the code but not *see what behaviour* it pins down or why it exists.
- **Code duplication** across tests, which raises the maintenance cost every time the system changes.
- Tests **can only be read by developers** — the people who wrote the requirement cannot check the test against their intent.
- Tests **cannot be used as documentation**, so the system's specification and its tests drift apart over time.

BDD's answer is to raise the description of behaviour to domain language so that the *same artefact serves three audiences at once* — the developer (it runs and goes red on a regression), the tester (it is the acceptance

criterion), and the domain expert (it is readable documentation). That triple-duty is precisely why BDD is the natural fit for the **Verification** phase: the scenarios *are* the acceptance criteria the phase must check, and "executed like normal tests" means that checking is automated and repeatable on every future change [Raj13] .

When & how it's applied. Behaviour is captured *before* the code, as a Given/When/Then scenario agreed between expert and developer (e.g. the pancake scenario on BDD p.5), then implemented in JGiven and asserted with AssertJ, then run continuously in CI. (Background reading: Dan North coined BDD as an evolution of TDD that focuses on *behaviour* and ubiquitous language — [Nor06] .)

Given / When / Then (the Gherkin-style scenario)

What it is. Given/When/Then is the canonical three-part template every BDD scenario is written in. It splits a behaviour into its starting context, the action under test, and the expected result — one structure, three clauses. The deck illustrates it with its running **pancake** example (BDD p.5):

```
Scenario: a pancake can be fried out of an egg milk and flour
```

```
Given an egg
  And some milk
  And the ingredient flour
When the cook mangles everything to a dough
  And the cook fries the dough in a pan
Then the resulting meal is a pan cake
```

The meaning of each keyword:

- **Given** — the **pre-conditions** / **starting context** that must hold before the behaviour runs ("an egg", "some milk", "the ingredient flour"). It describes *the world as it is set up*, not an action. Multiple givens are chained with **And**.
- **When** — the **single action** / **event** that exercises the behaviour under test ("the cook mangles everything to a dough", "the cook fries the dough in a pan"). This is the *trigger*; ideally a scenario has one logical When.
- **Then** — the **expected outcome** / **assertion** that must be true after the When ("the resulting meal is a pan cake"). This is where the test actually checks something, and in JGiven it is where AssertJ lives.

What it's used for / why it matters. GWT solves the "the point of the test is hard to grasp" problem (BDD p.3) by forcing every test to declare, in order, *what was assumed*, *what happened*, and *what should result*. Because the three parts read as a sentence, the scenario is simultaneously a requirement a domain expert can sanity-check and an executable acceptance criterion the Verification phase can run. It is the lingua franca of BDD: in plain-text tools it is the **Gherkin** language; in JGiven it is expressed directly in Java (see below).

When & how it's applied. You write one GWT scenario per concrete behaviour, mapping a user story onto it (BDDLab p.1). When the scenario runs, JGiven produces output that reads identically to the specification (BDD p.17) — which is what "living documentation" means in practice.

Relationship to TDD. TDD ("test-first") drives *design* from the inside out with developer-facing unit tests; BDD reframes the same test-first discipline around *externally observable behaviour* described in **ubiquitous/domain language** shared with non-developers ([Nor06]). BDD is sometimes summarised as "TDD done with the customer's vocabulary." The deck does not state this contrast on a slide, so treat it as reading-level context, not a deck claim.

Ubiquitous / domain language and "executable specifications"

What it is. Ubiquitous (domain) language is the shared vocabulary that domain experts and developers agree to use for the system — the same words for the same concepts in conversation, in scenarios, and ideally in the code and assertions too. An **executable specification** is what you get when a behaviour written in that language is *also runnable as a test*: the document and the test are one and the same file.

What it's used for / why it matters. The whole BDD value proposition rests on writing behaviour in the domain's own language so domain experts can read it (BDD p.4) and so developers, testers, and experts stop talking past each other. Making that domain-language description executable removes the classic failure mode of written specifications — drift. Prose documentation silently goes stale; an executable spec *cannot*, because a stale one **fails** when run and that failure is the alarm. This is the mechanism behind the deck's "living documentation" claim (BDD p.4) and is reinforced by the JGiven reports (BDD p.17–18), where the executed scenario literally *is* the readable document.

When & how it's applied. You phrase scenario steps in the domain's words (`an_egg` , `the_cook_fries_the_dough_in_a_pan`) rather than technical ones, and — when it matters — push that vocabulary all the way into the assertions via custom AssertJ assertions like `assertThat(student).isInMiddleSchool()` (BDD p.28), so even the Then step speaks the domain.

"Classical" plain-text BDD frameworks — and their maintenance cost

What they are. "Classical" BDD frameworks keep the scenario in a **separate plain-text or markup file** and connect it to Java through hand-written **step-definition** methods. The deck's roster (BDD p.6):

- **Cucumber** — Plain Text (Gherkin `.feature` files) + Java.
- **JBehave** — Plain Text + Java.
- **Concordion** — HTML + Java.
- **Fitness** ("Fitness" on the slide) — Wiki + Java.
- **robotframework.org** (RobotFramework).

What they're used for / why they matter — and their cost. Their selling point is that the scenario file is pure prose/markup, so in principle a non-developer could author it. The deck's verdict on the whole class, however, is one blunt phrase: **"Additional Maintenance Cost"** (BDD p.6). The reason (implicit, standard BDD knowledge): every scenario line in the plain-text file must be wired to a separately-maintained step-definition method, usually via regular-expression "glue". You therefore maintain *two artefacts in two languages* that must stay in lockstep — rename a step in the `.feature` file and the matching regex breaks; the indirection is brittle and hard to refactor with IDE tooling. In a maintenance context this is exactly the wrong property: expensive-to-maintain verification gets neglected, and neglected verification stops protecting the system.

When & how it's applied. You would reach for a classical framework when authorship by non-developers genuinely outweighs the glue cost. Otherwise the deck steers you to the next class, whose entire purpose is to remove that cost — relevant to the Verification phase because cheap-to-keep tests are the ones that actually stay green over a system's life.

Developer-friendly BDD frameworks

What they are. "Developer-friendly" BDD frameworks write the scenario in the **same language as the production code**, with no separate plain-text file and no regex glue. The scenario, the step implementations, and the system under test all live in one language and one toolchain. The deck's list (BDD p.7):

- **Spock** — Groovy.
- **ScalaTest** — Scala.
- **Jnario** — Xtend.
- **Serenity** — Java (starred on the slide).
- **JGiven** — Java (the deck's chosen tool).

What they're used for / why they matter. They exist to kill the "additional maintenance cost" of the classical class. Because scenarios are ordinary code, you get full IDE support — refactoring, autocomplete, compile-time checking, "find usages" — and there is only *one* artefact to keep in sync, so the test net stays maintainable as the system evolves. The trade-off, made explicit later for JGiven (BDD p.20), is that living in a programming language means non-developers can read the resulting reports but cannot author the scenarios.

When & how it's applied. These are the default choice when the team writing the scenarios is the development team — which, in this course's maintenance setting, it is. The deck commits to **JGiven** for the rest of the BDD section.

JGiven — developer-friendly BDD in pure Java

What it is. **JGiven** is an open-source (Apache 2) developer-friendly BDD framework for Java in which a scenario is **just a JUnit or TestNG test method** whose body is written so the method calls *read* like Given/When/Then. There is no `.feature` file: the prose and the executable test are the same Java. Its pitch (BDD p.9):

- **Developer friendly** — low maintenance overhead.
- **Readable test code** — Given-When-Then expressed directly in Java.
- **Modular and reusable** test code.
- **Reports for domain experts.**
- **Open source** (<http://jgiven.org>).

What it's used for / why it matters. JGiven is the tool that operationalises BDD's "executed like normal tests" property on the JVM while keeping the readability of plain-text BDD — without paying the classical frameworks' two-file maintenance tax. It is the Verification-phase workhorse of the deck: it turns the agreed GWT scenario into a runnable, CI-able acceptance test and simultaneously into a domain-readable report.

When & how it's applied. The defining trick is that a JGiven scenario is a normal test method whose calls mirror the plain-text scenario. The pancake scenario in JGiven (BDD p.10):

```
@Test
public void a_pancake_can_be_fried_out_of_an_egg_milk_and_flour() {
    given().an_egg().
        and().some_milk().
        and().the_ingredient( "flour" );

    when().the_cook_mangles_everything_to_a_dough().
        and().the_cook_fries_the_dough_in_a_pan();

    then().the_resulting_meal_is_a_pancake();
}
```

Note how `given()`, `when()`, `then()`, `and()` and the snake_case step methods reproduce the plain-text scenario *as Java* — so there is **no separate .feature file to keep in sync** (this is how JGiven removes the

"additional maintenance cost" of the classical frameworks). The test-method name and parameters even feed the generated report, so the report reads back in the same words as the test.

JGiven stage classes — modularity and reuse

What they are. A **stage class** is an ordinary Java class (extending JGiven's `Stage<SELF>` base) that holds the step methods for one part of a scenario. JGiven scenarios are *built from* these stage classes (BDD p.11):

- Stage classes provide **modularity** and **reuse**.
- Stage classes are a **unique feature of JGiven**, not present in any other BDD framework.
- **Typically one stage class is used for each of the Given, When, or Then** steps of a scenario.

What they're used for / why they matter. Stages are how JGiven keeps test code DRY and maintainable. By packaging the step methods into reusable classes, a single step (say, "build a drawing with one rectangle") is written once and reused across every scenario that needs it — directly attacking the "code duplication" test issue (BDD p.3) and underpinning JGiven's "highly modular and reusable test code" claim (BDD p.20). The one-stage-per-G/W/T convention also keeps each class focused: a Given stage only *sets up*, a When stage only *acts*, a Then stage only *asserts*, mirroring the GWT structure in the code's architecture.

When & how it's applied. The pancake scenario is implemented by (at least) three stages — a Given stage, a When stage, and a Then stage — each a Java class extending `Stage<SELF>`. The self-type generic (`Stage<GivenIngredients>`) is what lets each step method end with `return this;` and so chain fluently (`given().an_egg().and()....`). Because stages are ordinary classes, their step methods are reusable across many scenarios.

JGiven state transfer between stages (@ScenarioState and friends)

What it is. Stages must pass data down the Given → When → Then pipeline (the ingredients built in Given have to be visible to When; the meal produced in When has to reach Then). JGiven does this with **annotated fields** rather than method parameters or shared globals (BDD p.13):

- **Fields can be annotated with @ScenarioState .**
- **Values are written and read between stages** — JGiven copies the field value out of the stage that produced it into the same-typed field of the next stage.
- **@ProvidedScenarioState** and **@ExpectedScenarioState** are the *directional* alternatives:
- **@ProvidedScenarioState** — this stage **produces / provides** the value (it writes it out for later stages). *Provided = producer.*
- **@ExpectedScenarioState** — this stage **consumes / expects** a value provided by an earlier stage. *Expected = consumer.*
- **@ScenarioState** — **bidirectional** (the field is both read and written).

What it's used for / why it matters. State transfer is the mechanism that lets you split a scenario across multiple reusable stage classes *without* coupling them through explicit parameter wiring. Each stage just declares the fields it provides or expects, and JGiven matches them up by type behind the scenes, so stages stay independent and reusable while still cooperating on one scenario's data flow. The directional annotations also document intent — reading a stage tells you exactly which state it consumes and which it produces.

When & how it's applied. A field is marked `@ProvidedScenarioState` in the stage that creates the value and `@ExpectedScenarioState` in the stage that uses it; JGiven copies it across at scenario-execution time. The state-transfer diagram (BDD p.12) visualises this as three horizontal bands — **Given Stage** (green), **When Stage** (blue), **Then Stage** (orange) — with state objects (`state1`, `another state` → `state 2`, `result`)

flowing *downward*: state produced in Given flows into When; the When stage performs `some action` producing a `result`; and the Then stage receives `state1`, `result`, and `state 2` to run `some assertion`. That picture is the GWT data pipeline the annotations wire up.

JGiven worked stage classes (the pancake example, in code)

What it is. This is the canonical "how JGiven actually works" example: the three concrete stage classes that implement the pancake scenario, showing the annotations, the `return this` chaining, and the AssertJ assertions all working together.

What it's used for / why it matters. It demonstrates, end to end, how a readable Java scenario decomposes into reusable stages and how state flows between them via the annotations — the single example most likely to be asked to be read or reproduced in the exam.

When & how it's applied. The deck shows the three stages in sequence.

Given stage — `GivenIngredients` (BDD p.14): extends `Stage<GivenIngredients>`, holds a `@ProvidedScenarioState List<String> ingredients` (it *provides* the ingredient list downstream), and its step methods (`an_egg()`, `some_milk()`, `the_ingredient(String)`) add to that list and `return this` for fluent chaining:

```
public class GivenIngredients extends Stage<GivenIngredients> {

    @ProvidedScenarioState
    List<String> ingredients = new ArrayList<String>();

    public GivenIngredients an_egg() {
        return the_ingredient( "egg" );
    }

    public GivenIngredients the_ingredient( String ingredient ) {
        ingredients.add( ingredient );
        return this;
    }

    public GivenIngredients some_milk() {
        return the_ingredient( "milk" );
    }
}
```

When stage — `WhenCook` (BDD p.15): pulls in a collaborator and the upstream state, performs the action, and provides downstream state. It *expects* `ingredients` (provided by Given) and *provides* `dough` and `meal`:

```
public class WhenCook extends Stage<WhenCook> {
    @Autowired
    @ScenarioState
    Cook cook;

    @ExpectedScenarioState
    List<String> ingredients;           // provided by the Given stage

    @ProvidedScenarioState
    Set<String> dough;

    @ProvidedScenarioState
    String meal;

    public WhenCook the_cook_fries_the_dough_in_a_pan() {
```



```

        assertThat( cook ).isNotNull();
        assertThat( dough ).isNotNull();

        meal = cook.fryDoughInAPan( dough );
        return this;
    }
}

```

Note: the When stage already uses **AssertJ** `assertThat(...).isNotNull()` as sanity checks, foreshadowing the AssertJ half of the deck. The `@Autowired` shows JGiven integrating with Spring dependency injection (the `Cook` collaborator is injected by Spring, and `@ScenarioState` also makes it available to other stages).

Then stage — `ThenMeal` (BDD p.16): consumes the produced `meal` (it *expects* it) and asserts the outcome with AssertJ. Its step methods are `void` because nothing chains after a Then:

```

package com.tngtech.jgiven.examples.pancakes.test.steps;

import static org.assertj.core.api.Assertions.assertThat;
import com.tngtech.jgiven.Stage;
import com.tngtech.jgiven.annotation.ExpectedScenarioState;

public class ThenMeal extends Stage<ThenMeal> {
    @ExpectedScenarioState
    String meal;

    public void the_resulting_meal_is_a_pan_cake() {
        the_resulting_meal_is_a( /* expectedMeal: */ "pancake" );
    }

    public void the_resulting_meal_is_a( String expectedMeal ) {
        assertThat( meal ).isEqualTo( expectedMeal );
    }
}

```

This trio is the canonical example: **Given provides** the ingredients, **When consumes** them + a `Cook`, does the action and **provides** the `meal`, and **Then consumes** the `meal` and **asserts** it with AssertJ. The whole pipeline is wired by the `@...ScenarioState` annotations — no explicit parameter passing.

Line-by-line commentary on the three stage classes (BDD p.14–16)

This subsection slows the worked example down to the level an exam marker (or an oral examiner) can probe: what each construct on the three code slides *does* and *why it is there*.

The class declaration and the self-type generic. Every stage declares itself as `public class GivenIngredients extends Stage<GivenIngredients>` (BDD p.14), `WhenCook extends Stage<WhenCook>` (BDD p.15), `ThenMeal extends Stage<ThenMeal>` (BDD p.16). The generic parameter is the class *itself* — the "self-type" idiom. Its purpose: the inherited fluent infrastructure can be typed to return *the concrete stage*, so a chain like `given().an_egg().and().some_milk()` stays statically typed as `GivenIngredients` all the way through, and the compiler offers exactly that stage's step methods after every dot. Without the self-type, `return this` would come back as a bare `Stage` and the chain would lose its vocabulary.

Field initialisation in the Given stage. `List<String> ingredients = new ArrayList<String>();` (BDD p.14) — the Given stage begins with an *empty world* and each Given step adds to it. This is the code-level meaning of "Given = pre-conditions": the stage literally accumulates the starting context, then exports it via `@ProvidedScenarioState`.

Intra-stage step delegation (a reuse pattern within one stage). `an_egg()` does nothing but `return the_ingredient("egg");` and `some_milk()` returns `the_ingredient("milk")` (BDD p.14). One *parameterised* step — `the_ingredient(String)` — is the single point of truth that mutates state; the domain-named wrappers exist purely for scenario readability. The Then stage repeats the identical pattern: `the_resulting_meal_is_a_pan_cake()` simply calls the parameterised `the_resulting_meal_is_a("pancake")`, which holds the one real assertion `assertThat(meal).isEqualTo(expectedMeal)` (BDD p.16). Exam phrasing: *step methods may call other step methods*, giving reuse not only **across** scenarios (via stages, BDD p.11) but **inside** a single stage.

Why snake_case method names. JGiven derives the report prose from the method names — underscores become spaces. Compare the method `the_cook_fries_the_dough_in_a_pan()` (BDD p.10) with the console line `And the cook fries the dough in a pan` (BDD p.17): the test code is the report text. Method **arguments** are woven into the prose the same way: `the_ingredient("flour")` (BDD p.10) prints as `And the ingredient flour` (BDD p.17). This is the concrete mechanism behind "readable test code (Given-When-Then)" (BDD p.9).

The When stage's four annotated fields are the whole state mechanism in one screen. `WhenCook` (BDD p.15) shows, in a single class: `@Autowired @ScenarioState Cook cook;` (a Spring-injected collaborator that is *also* shared bidirectionally as scenario state), `@ExpectedScenarioState List<String> ingredients;` (consumed from the Given stage), and two `@ProvidedScenarioState` fields `Set<String> dough;` and `String meal;` (produced for downstream stages). If you can explain those four fields, you can explain JGiven state transfer completely (BDD p.13).

The sanity assertions inside the When step. `the_cook_fries_the_dough_in_a_pan()` opens with `assertThat(cook).isNotNull();` and `assertThat(dough).isNotNull();` before doing `meal = cook.fryDoughInAPan(dough);` (BDD p.15). These AssertJ guards fail fast, with a readable message, if the scenario wiring is broken (no `Cook` injected; no dough produced by the earlier mangling step) — much clearer than a `NullPointerException` deep inside `fryDoughInAPan`. Note also the division of labour: the stage stays *thin* and delegates the real work to the production object `Cook` — the When step orchestrates, the system under test acts.

An honest gap on the slide. `dough` is declared `@ProvidedScenarioState` in `WhenCook` (BDD p.15) and is asserted non-null before frying, which implies the *earlier* When step from the scenario — `the_cook_mangles_everything_to_a_dough()` (BDD p.10) — created it. That mangling step's body is **not shown** on any slide; only the frying step is. If an exam asks you to reproduce the example, reproduce what the slides show and note that the mangling step (which fills `dough`) is elided.

The Then stage's imports tell their own story. Slide p.16 shows the full file header: `package com.tngtech.jgiven.examples.pancakes.test.steps;`, a **static import** `import static org.assertj.core.api.Assertions.assertThat;` (this is what lets the code write bare `assertThat(...)`), plus only two JGiven imports — `com.tngtech.jgiven.Stage` and `com.tngtech.jgiven.annotation.ExpectedScenarioState`. Two take-aways: (1) the JGiven API surface a Then stage needs is tiny; (2) the package prefix `com.tngtech` reveals that the pancake example ships from TNG Technology Consulting's own JGiven repository — the same TNG whose industrial experience report appears on slide 19. The deck's example and its evidence base come from the same source.

Why Then steps are void. Both `the_resulting_meal_is_a_pan_cake()` and `the_resulting_meal_is_a(String)` return `void` (BDD p.16), whereas every Given/When step returns the stage type. Nothing chains *after* a terminal assertion, so there is no need for `return this` — and a `void` signature documents "this is a leaf of the scenario."

One rendering artefact to not be fooled by. On slide p.16 the call `the_resulting_meal_is_a(expectedMeal: "pancake" );` appears to contain a label `expectedMeal:` — that grey text is an **IDE parameter-name inlay hint**, not Java syntax. The actual source is `the_resulting_meal_is_a("pancake");`. Java has no named arguments; reproduce the call without the hint.

JGiven reports — console and HTML5 (living documentation)

What it is. When a JGiven scenario runs, JGiven generates two reports that render the executed scenario back as readable prose: a plain-text **console report** and a browsable **HTML5 app report**. They are the tangible output of BDD's "living documentation" promise — the test run *becomes* the document.

What it's used for / why it matters. These reports are how JGiven delivers on the "reports for domain experts" and "living documentation" pledges (BDD p.9, p.4): a domain expert who cannot read Java can still read the report and confirm the behaviour is what they asked for, and because the report is regenerated from the actual test execution it can never silently drift from the code. They are also the Verification-phase pass/fail signal: the change is verified when the report is green.

When & how it's applied. The **console output** (BDD p.17) prints the test class and the scenario in plain Given/And/When/And/Then prose:

```
Test Class: com.tngtech.jgiven.examples.pancakes.test.SpringPanCakeScenarioTest
A pancake can be fried out of an egg milk and flour

    Given an egg
      And some milk
      And the ingredient flour
    When the cook mangles everthing to a dough
      And the cook fries the dough in a pan
    Then the resulting meal is a pan cake
```

The **HTML5 app report** (BDD p.18) is a browsable web report ("JGiven Report") with a left-hand summary (All Scenarios / Failed / Pending counts), tags, and classes; the screenshot shows "53 Successful, 0 Failed, 0 Pending, 53 Total", grouped/sortable scenario rows, and support for data tables, tags, AsciiDoc reports, German-language scenarios, etc. Verification of a behaviour is "passed" exactly when its row is green and the report shows 0 Failed / 0 Pending.

Inside the HTML5 report screenshot (BDD p.18)

The single screenshot on slide 18 carries far more exam-usable detail than its caption suggests; here is the full inventory of what is visible.

Chrome and navigation. The page is titled "**JGiven Report**" (green header bar) with a **search-in-scenarios** box at top right. The left sidebar has three navigation blocks: **SUMMARY** (All Scenarios — 53, Failed Scenarios — 0, Pending Scenarios — 0), **TAGS** (the legible expandable entries are *BrowserTest*, *Features*, *Issue*), and **CLASSES** (a package tree rooted at `com`). The main pane is headed "**All Scenarios**" with the aggregate line "**53 Successful, 0 Failed, 0 Pending, 53 Total (0.077s)**", a green status doughnut, pagination, and a toolbar of dropdowns: **Group By, Sort By, Status, Tags, Classes** (BDD p.18).

The three-valued status model. Note that the report counts **Pending** scenarios as a first-class third state alongside Successful and Failed (BDD p.18). Pending means a scenario is declared but its implementation is not (yet) executed — useful in BDD's write-the-scenario-first workflow, where acceptance criteria exist before the code that satisfies them. An exam answer that says JGiven reports are "pass/fail" is incomplete; they are **pass / fail / pending**.

The visible scenario rows (JGiven dogfooding). Every row in the screenshot is a scenario from JGiven's *own* test suite — JGiven verifying JGiven, which is itself a nice illustration of living documentation. Legible groups and titles include (BDD p.18):

- **Argument Analyzer** — "Different structure prevent data table", "Multiple formatted arguments lead to one parameter", "Multiple parameter usages lead to one parameter" (rows carry numbered *Issue* tag chips; the exact issue numbers are too small to read reliably in the slide).
- **Ascii Doc Report Generator** — "The AsciiDoc reporter generates an index file" (*AsciiDoc Report* tag) — evidence that JGiven can emit AsciiDoc reports besides console and HTML5.
- **Data Provider Test Ng** — "A scenario with one failing case still executes the following ones" (*TestNG* tag) and **Data Provider** — "A scenario with one failing case leads to a failed scenario" (*JUnit* tag) — together confirming the JUnit *and* TestNG integrations claimed on the summary slide (BDD p.20), and showing parameterised scenarios with multiple cases.
- **De Szenario** — "Szenarien können in deutsch geschrieben werden" (*German Scenarios* tag) — scenarios can be written in German: the GWT prose mechanism is natural-language-agnostic, since report text is just derived from method names.
- **Difference Analyzer** — four scenarios: the difference analyzer "should find additional steps at the beginning", "should find additional steps at the end", "should find additional steps in the middle", and "should find differences in step arguments" (*Case Diffs* tag).
- **HTML App** — "Attachments appear in the HTML5 report", "Attachments of all cases appear in the HTML5 report when having a data table", "Clicking on tag labels opens the tag page", "Navigation links of the HTML report can be customized using a custom JS file" (*BrowserTest*, *HTML5 Report*, *Attachments*, *Tags* chips).

Each row shows a green check, a per-scenario **duration** (milliseconds to seconds), and clickable **tag chips**; clicking a tag label opens that tag's page (that behaviour is itself one of the listed scenarios). Feature support you can read straight off the rows: **data tables**, **attachments**, **tags**, **AsciiDoc output**, **custom JS report navigation**, **non-English scenarios**, **JUnit + TestNG data providers** (BDD p.18).

Why this matters for Verification. The report is the Verification phase's human-facing dashboard: a domain expert (or examiner) can confirm at a glance that all acceptance criteria are green, drill in by tag (e.g. every scenario touching one feature), by class, or by free-text search — without reading a line of Java. That is "reports for domain experts" (BDD p.9) made concrete.

AssertJ documentation pointer on the basic-use slide

Slide 24 closes its bullet list with an explicit reference: "**See more <http://joel-costigliola.github.io/assertj>**" (BDD p.24) — the AssertJ project documentation site (Joel Costigliola is AssertJ's creator). Together with the final slide's "**The one minute starting guide**" and "**The getting started guide**" pointers (BDD p.30), these are the deck's two on-ramps for actually adopting AssertJ in the lab portfolio.

JGiven in industry — the TNG experience report

What it is. A real industrial experience report included in the deck (BDD p.19, "TNG Practical Experience") — empirical evidence from the company behind JGiven (TNG Technology Consulting) rather than a vendor claim.

What it's used for / why it matters. It is the deck's evidence base for the developer-friendliness and low-maintenance claims: a concrete, citeable data point you can use in an exam to argue "BDD-in-code

measurably lowers test-maintenance cost and scales to large teams." Note the honest caveat that the maintenance-cost reduction came with "no hard numbers" — useful for a balanced answer.

When & how it's applied. Cite it when asked for evidence. The headline figures (BDD p.19):

- **3 years** of experience on a **large Java Enterprise project** (up to **70 developers**).
- **Over 3000 scenarios**.
- Readability and reusability of test code **greatly improved**.
- Maintenance costs of automated tests **reduced** (but "no hard numbers").
- **Well accepted** by developers and **easy to learn** by new developers.
- Developers and domain experts **collaborate using scenarios**.

JGiven summary — strengths and the one big limitation

What it is. The deck's wrap-up of JGiven (BDD p.20): a list of its strengths plus the single decisive caveat that defines its trade-off space.

What it's used for / why it matters. This is the slide for an evaluation/"discuss the trade-offs" exam question — it tells you both why you would choose JGiven and the one reason you might not.

When & how it's applied. The strengths and the catch (BDD p.20):

- Developer friendly.
- Highly modular and reusable test code.
- **Just Java** — no further language needed.
- Easy to integrate into existing test infrastructures (**JUnit**, **TestNG**).
- Open source (**Apache 2 licence**).
- **Maven and Jenkins plugins** (CI integration — this is what ties Verification to the Conclusion/CI phase, so scenarios run automatically on every build).
- Nice reports for domain experts.
- **Domain experts CANNOT write scenarios in JGiven** ← the key trade-off.

That last bullet is the central tension of the deck: JGiven buys low maintenance and reuse by living in Java, but the price is that **non-developers can read the reports yet cannot author the scenarios** — unlike classical plain-text Cucumber/Gherkin, where a domain expert could in principle write the `.feature` file. Expect an exam question on this trade-off.

AssertJ — fluent, readable assertions (the Then-step engine)

What it is. **AssertJ** is an actively-maintained Java assertion library with a *fluent* API: you start with `assertThat(actual)` and chain readable, sentence-like checks onto it. It is the second half of the deck (BDD p.21–30) and the assertion engine that runs *inside* the Then steps of a JGiven scenario.

What it's used for / why it matters. AssertJ exists because JUnit's built-in assertions were, in the deck's words, "underpowered from the start," which drove developers to bolt on **Hamcrest** and **Fest** and produced "a confusion of JUnit, Hamcrest and Fest" (BDD p.22). It solves that by being one coherent, readable, near-complete superset that replaces all three. Its readability is the point that matters for BDD: a Then step that reads like a sentence keeps the whole scenario aligned with the "living documentation" goal. The deck's specific complaints (BDD p.22):

- **JUnit** is very simplistic.

- **Fest** looks to be abandonware (unmaintained).
- **Hamcrest** is stagnant and ugly.

Why AssertJ (BDD p.23): it is still actively maintained; a **near-complete superset** of the others' functionality; well designed; and easy to get started with, to enhance, and **to read**.

When & how it's applied. AssertJ is used wherever a check is made — most visibly in JGiven Then stages (`assertThat(meal).isEqualTo("pancake")`). Its core machinery (BDD p.24):

- `Assertions.assertThat()` — **type-specific factory methods** (overloaded so you get the right assertion object for whatever type you pass).
- `AbstractAssert` — the base class of all the **type-specific assertions**.
- a **Fluent API** built on **good use of polymorphism**.

Basic use example (BDD p.25):

```
@Test
public void shouldProvideAnExample() {
    String actual = "This is a test";
    assertThat(actual).contains("is")
                    .startsWith("This");

    String[] actualArray = new String[] {"This", "is", "a", "test"};
    assertThat(actualArray).contains("is")
                        .startsWith("This");
}
```

Take-aways (BDD p.25): the `assertThat` factory **yields a String- or Array-assert as needed**; `contains` and `startsWith` are **polymorphic** (the same method names work for both String and array); and the **fluent style** lets you chain `.contains().startsWith()` in one statement. This readability is exactly why AssertJ pairs so well with BDD's "living documentation" goal — the assertion reads like a sentence.

AssertJ custom Condition (reusable predicates)

What it is. A **Condition** is a small reusable AssertJ predicate object: you subclass `Condition<T>` and override its `matches(T)` method to return true/false, packaging a bit of pass/fail logic as a named, reusable unit (a *functor*). The deck (BDD p.26):

- Custom conditions allow **more elaborate tests** than the built-ins.
- They can be applied to types that **already have a custom assertion** in a straightforward way.

What it's used for / why it matters. Conditions exist so that bespoke pass/fail logic — anything beyond AssertJ's out-of-the-box checks — can be written once, given a meaningful name, and reused across many assertions instead of being copy-pasted as inline boolean expressions. They are AssertJ's answer to the Hamcrest matcher, but expressed in AssertJ's fluent style, and they keep the Then step readable (`is(evenDivBySix)` reads better than an inline modulo check).

When & how it's applied. Implement the `Condition` (override `matches`), then apply it with the fluent `.is(...)` / `.isNot(...)` methods (BDD p.27):

```
@Test
public void shouldBeEvenlyDivisibleBySix() {
    Condition<Integer> evenDivBySix = new Condition<Integer>() {
        @Override
```

```

    public boolean matches(Integer value) {
        return (value % 6) == 0;
    }
};
assertThat(12).is(evenDivBySix);
assertThat(8).isNot(evenDivBySix);
}

```

The same `evenDivBySix` condition can now be reused anywhere an `Integer` is asserted, in both its positive (`.is`) and negative (`.isNot`) forms.

AssertJ custom assertions (domain-specific `assertThat`)

What it is. A **custom assertion** is a domain-specific extension to AssertJ's fluent API: you create your own assertion type (e.g. `StudentAssert`) with methods named in your domain's vocabulary (e.g. `isInMiddleSchool()`) and your own `assertThat` factory that returns it, so you can write `assertThat(student).isInMiddleSchool()` exactly as if it were built in.

What it's used for / why it matters. Whereas a `Condition` packages a one-off predicate, a custom assertion gives a *whole domain type* its own first-class assertion vocabulary. This is how you push **ubiquitous/domain language all the way into the assertion API**, so even the Then step reads in the domain's words — the assertion-level counterpart of BDD's domain-language scenarios, and the deepest expression of the "readable by domain experts" goal.

When & how it's applied. The recipe (BDD p.28), using the deck's `Student / isInMiddleSchool()` motivating example:

1. **Subclass `AbstractAssert`** for `Student` (e.g. `StudentAssert`) and implement the domain method `isInMiddleSchool()`.
2. **Subclass `Assertions`** to add a new `assertThat` factory method that returns your `StudentAssert`.
3. **Use your new factory and assertions** exactly as you always would:
`assertThat(student).isInMiddleSchool()`.

AssertJ-Swing — verifying GUIs (relevant to JHotDraw-style editors)

What it is. **AssertJ-Swing** is the AssertJ module for automating tests of **Swing GUI** applications — it drives the actual widgets (buttons, canvases, menus) as a user would and then asserts on the resulting UI state. The deck's feature list (BDD p.29):

- **Simulation of user interaction** with a GUI (e.g. **drag 'n drop**).
- **Reliable GUI component lookup** — by type, by name, or by custom search criteria.
- Support for **all Swing components** included in the JDK.
- **Compact and powerful API** for creating and maintaining **functional GUI tests**.
- Ability to **embed screenshots of failed GUI tests** in HTML test reports.
- Can be used with either **TestNG** or **JUnit**.
- Supports testing **violations of Swing's threading rules** (the EDT — Event Dispatch Thread).

What it's used for / why it matters. Plain assertions can only check objects and return values; AssertJ-Swing lets the Verification phase reach the *GUI layer*, exercising behaviour that only manifests through real user interaction (dragging a figure, clicking a tool). Its reliable component lookup is what makes such tests stable across UI refactors instead of brittle; the screenshot-on-failure feature gives a visual diagnosis of a failed GUI test inside the same HTML report; and the EDT-violation checks catch a whole class of subtle

Swing concurrency bugs (mutating the GUI off the Event Dispatch Thread) that are otherwise invisible until they cause intermittent corruption.

When & how it's applied. It is the tool for verifying a Swing application's behaviour at the GUI level. This matters for the course's running case study: JHotDraw is a **Swing** drawing editor, so AssertJ-Swing is what you would use to verify a JHotDraw change at the GUI level — drag a figure, drop it, and assert the canvas state, embedding a screenshot if it fails. The lab makes this explicit: "For Swing applications use the AssertJ-swing to automate the Scenarios" (BDDLab p.1).

Getting started

What it is. The closing slide is a pointer to AssertJ's own onboarding documentation rather than new technical content.

What it's used for. It tells you where to go to start using AssertJ in practice for the lab portfolio.

When & how it's applied. Follow "The one minute starting guide" and "The getting started guide" on AssertJ's site when setting up your own assertions (BDD p.30).

Where BDD sits in the change process (course frame)

What it is. This section locates BDD inside the eight-phase software-change process. It is *course-frame synthesis* of the deck with [Raj13], not a deck slide — the BDD deck itself never names the eight phases.

What it's used for / why it matters. It answers the exam-favourite "where does BDD fit in the change process?" by tying the tool-focused deck back to the course's process model: BDD is how the **Verification** phase is operationalised, and the phase is "passed" when the scenarios are green.

When & how it's applied. The eight-phase process ends in **Verification**, which "confirms the change is correct and complete: tests pass, **acceptance criteria (e.g. BDD scenarios)** are met, and no regressions were introduced" (course overview). BDD operationalises Verification by making those acceptance criteria **executable**. The connections to the neighbouring phases:

- **Initiation** produces the change request / user story; BDD turns that story into Given/When/Then **before** coding (BDDLab p.1), so the acceptance criterion exists up front and the team agrees on "done" before writing code.
- **Actualization** writes the production code; the BDD scenarios are the target it must satisfy (the scenario goes from red to green as the change is implemented).
- **Conclusion / CI** runs the scenarios automatically (JGiven's Maven/Jenkins plugins, BDD p.20) so every integration is verified, not just the developer's local run.
- **Verification** is "passed" exactly when all scenarios go green and the HTML report shows 0 Failed / 0 Pending (BDD p.18).

(This phase mapping is course-frame synthesis; the BDD deck itself does not name the eight phases.)

JHotDraw Connection

Deck grounding: The BDD deck and lab **do not mention JHotDraw**. The link below is course-frame synthesis ([Raj13] + course overview), provided because the exam expects you to connect every lecture back to the running case study.

JHotDraw is the course's canonical case study: an open-source **Java Swing** GUI framework for structured drawing editors (figures, connectors, a canvas), prized as a pattern-rich, maintainable design. When you perform a change on JHotDraw following the eight-phase process, **Verification is where Lecture 9's tooling applies**:

- **A change to JHotDraw begins as a request/user story.** Following the lab's recipe (BDDLab p.1), you first express the desired behaviour as a Given/When/Then scenario — e.g. *Given a drawing with one rectangle, When the user drags the rectangle 50px right, Then the rectangle's x-coordinate increases by 50*. This is the acceptance criterion the change must satisfy, agreed before coding.
- **JGiven implements the scenario in pure Java** (BDD p.10–16), with a Given stage that builds the drawing (`@ProvidedScenarioState` the figure), a When stage that performs the drag (consuming the figure with `@ExpectedScenarioState` and providing the resulting figure state), and a Then stage that asserts the new coordinates with AssertJ. Because JHotDraw is large and real, JGiven's **stage reuse** (BDD p.11) pays off: a `GivenADrawing` stage is reused across many figure scenarios, so duplication stays low as the test suite grows.
- **AssertJ-Swing drives JHotDraw's actual GUI** (BDD p.29): simulate the **drag 'n drop** on the real canvas, look up the canvas component reliably, assert on it, and **embed a screenshot** if the assertion fails — and it can test **Swing threading-rule (EDT) violations**, which a graphics framework like JHotDraw, with its constant view repainting, is especially prone to.
- **The JGiven HTML5 report** (BDD p.18) becomes living documentation of JHotDraw's verified behaviours — readable by a domain expert, even though (per the JGiven limitation, BDD p.20) only developers can author the scenarios.
- **Design-pattern angle** (`[GHJV94]`): JHotDraw is built on GoF patterns (Composite figures, Observer for change notification, Strategy for tools). BDD scenarios verify the *behaviour those patterns produce* (e.g. an Observer correctly refreshing the view) without coupling the test to the pattern's internal structure — behaviour over implementation, exactly BDD's point, and the property that keeps the tests valid when the pattern's internals are refactored.

In short: **for any JHotDraw change, the BDD scenarios are the Verification-phase gate, JGiven makes them executable Java, and AssertJ/AssertJ-Swing assert the (often GUI) outcome.**

Sketch: a JGiven verification suite for a JHotDraw change (course-frame synthesis)

The following sketch is **synthesis** — it appears on no slide — but it is assembled *exclusively* from mechanics the deck does show (stage classes BDD p.11, state annotations BDD p.13–16, AssertJ assertions BDD p.25, AssertJ-Swing capabilities BDD p.29), transplanted onto the course's case study. It is the kind of transfer question an exam can ask: "apply the lecture's tooling to JHotDraw."

Scenario (acceptance criterion for a "move figure" change): *Given a drawing with one rectangle at (10, 10), When the user drags the rectangle 50 pixels right, Then the rectangle's x-coordinate is 60.*

```
@Test
public void a_rectangle_can_be_dragged_to_a_new_position() {
    given().a_drawing_with_one_rectangle_at( 10, 10 );
    when().the_user_draggs_the_rectangle_right_by( 50 );
    then().the_rectangle_x_coordinate_is( 60 );
}
```

Following the pancake pattern exactly: a `GivenADrawing` extends `Stage<GivenADrawing>` stage holds `@ProvidedScenarioState` fields for the drawing and the figure (the analogue of `ingredients`, BDD p.14); a `WhenUserDrags` extends `Stage<WhenUserDrags>` stage marks the figure `@ExpectedScenarioState`, performs

the drag, and `@ProvidedScenarioState` -exports the resulting figure state (the analogue of `WhenCook`'s consume-act-provide shape, BDD p.15); a `ThenFigure` extends `Stage<ThenFigure>` stage consumes the result with `@ExpectedScenarioState` and runs a `void` step containing `assertThat(figure.getX()).isEqualTo(expectedX)` (the analogue of `ThenMeal`, BDD p.16). Two implementation levels are available for the `When` step:

- **Model level** — call `JHotDraw`'s figure API directly (fast, headless; verifies the model behaviour but not the UI wiring).
- **GUI level** — use **AssertJ-Swing** to simulate the actual **drag 'n drop** on the canvas with reliable component lookup by type/name/custom criteria, and let a failed run **embed a screenshot** in the HTML report (BDD p.29). This level also exposes **EDT threading-rule violations** — `JHotDraw` repaints views constantly, so off-EDT mutations are a realistic bug class `AssertJ-Swing` can catch (BDD p.29).

The pay-off mirrors the deck's claims: `GivenADrawing` is written once and reused by every figure scenario (modularity/reuse, BDD p.11), the scenario reads as domain prose for the report (BDD p.17–18), and the green report is the change's Verification-phase exit ticket (course frame, [Raj13]).

Worked Example / Process Walkthrough

The deck's own running example is the **pancake scenario**, shown end-to-end across the plain-text spec, the `JGiven` test, the three stage classes, and the report. Walked through as a Verification exercise:

Step 0 — User story → scenario (lab method, BDDLab p.1). Behaviour is captured first in `Given/When/Then`. The deck's pancake scenario (BDD p.5):

```
Scenario: a pancake can be fried out of an egg milk and flour

Given an egg
  And some milk
  And the ingredient flour
When the cook mangles everything to a dough
  And the cook fries the dough in a pan
Then the resulting meal is a pan cake
```

Step 1 — Express it as a JGiven test (BDD p.10). The scenario becomes a JUnit test method whose calls mirror the prose (no separate `.feature` file):

```
@Test
public void a_pancake_can_be_fried_out_of_an_egg_milk_and_flour() {
    given().an_egg().and().some_milk().and().the_ingredient( "flour" );
    when().the_cook_mangles_everything_to_a_dough().and().the_cook_fries_the_dough_in_a_pan();
    then().the_resulting_meal_is_a_pancake();
}
```

Step 2 — Implement the three stages with state transfer (BDD p.14–16). - `GivenIngredients` `@ProvidedScenarioState` `List<String> ingredients; an_egg(), some_milk(), the_ingredient(String)` append and return this. (It provides the ingredients.) - `WhenCook` consumes `ingredients (@ExpectedScenarioState)`, uses an `@Autowired @ScenarioState Cook`, **provides** `Set<String> dough` and `String meal; the_cook_fries_the_dough_in_a_pan()` runs `assertThat(cook).isNotNull(); assertThat(dough).isNotNull(); meal = cook.fryDoughInAPan(dough);`.

- ThenMeal consumes meal (@ExpectedScenarioState); the_resulting_meal_is_a_pan_cake() calls the_resulting_meal_is_a("pancake"), which does assertThat(meal).isEqualTo(expectedMeal).

The state flows **Given** → **When** → **Then** exactly as the state-transfer diagram shows (BDD p.12): ingredients out of Given, into When; dough / meal out of When, into Then — all wired by the annotations, with no explicit parameter passing.

Step 3 — Run it; read the report (BDD p.17–18). The console prints the scenario back as prose (living documentation), and the HTML5 app shows it as a green, browsable row. Verification of this behaviour is "passed" when the row is green and the report shows 0 Failed.

Step 4 — The assertion layer is AssertJ (BDD p.25–28). The Then step's `assertThat(meal).isEqualTo("pancake")` is AssertJ's fluent API; for richer checks you would add a custom **Condition** (BDD p.27) or a domain-specific assertion like `assertThat(meal).isAPancake()` (the custom-assertion recipe, BDD p.28).

Mini-walkthrough from the lab (BDDLab p.1) — mapping a user story to a scenario, the calculator example:

User story: "As a calculator user, I want to add two numbers so that I can do addition." *BDD scenario:* **Given** I have two numbers 500 & 500 / **When** I add them up / **Then** I should get result 1000.

This is the lab's required first move for the portfolio: take each user story, write its Given/When/Then, automate with JGiven, assert with AssertJ (AssertJ-Swing for Swing) (BDDLab p.1).

The lab's other two Figure-1 mappings, analysed clause by clause (BDDLab p.1)

The lab's Figure 1 contains **three** complete user-story→scenario mappings, not just the calculator. All three are exam-reproducible:

Mapping 2 — the Math teacher (sorting).

User story: "As a Math teacher, I want to automate marks sorting process so that I can declare top 5 in my class." *BDD scenario:* **Given** a list of numbers / **When** I sort the list / **Then** the list will be in numerical order.

Clause analysis: the Given sets up the input data (a list of numbers — the pre-condition world); the When is the single action under test (sorting); the Then states the post-condition property (numerical order). Notice the abstraction step the mapping performs: the story's *motivation* ("declare top 5 in my class") does **not** appear in the scenario — the scenario verifies the *capability* (sorting works), not the reason it was wanted. That story-motivation-versus-scenario-check distinction is a useful thing to articulate if asked to perform a mapping yourself.

Mapping 3 — the QA engineer (smoke test).

User story: "As a QA engineer, I want to check a critical feature so that I can do smoke test easily." *BDD scenario:* **Given** I visit Google.com / **When** I type 'TestingWhiz' as a search string / **Then** I should get search results matching TestingWhiz.

Clause analysis: here the Given is a *navigation* pre-condition (being on a page), the When is a concrete user interaction (typing a search string), and the Then asserts on observable output (matching results). This row shows BDD applied at the **system/UI level** rather than the unit level — the same GWT structure spans both, which is exactly why the lab can tell Swing-app teams to automate their scenarios with **AssertJ-Swing** (BDDLab p.1): a UI-level When ("the user drags / types / clicks") needs a UI-driving tool.

Reading the three rows together: row 1 (calculator) is computation-level, row 2 (sorting) is data/algorithm-level, row 3 (search) is UI/system-level — Figure 1 quietly demonstrates that Given-When-Then is level-agnostic.

The calculator scenario implemented in JGiven (synthesis from deck patterns)

No slide implements the calculator story — this sketch is **synthesis**, but every construct in it is taken from the deck's pancake stages (BDD p.10, p.14–16) so you can practise the transfer. The lab's row 1 scenario — Given I have two numbers 500 & 500 / When I add them up / Then I should get result 1000 (BDDLab p.1) — becomes:

```
@Test
public void two_numbers_can_be_added() {
    given().two_numbers( 500, 500 );
    when().they_are_added_up();
    then().the_result_is( 1000 );
}
```

- `GivenNumbers` extends `Stage<GivenNumbers>` holds `@ProvidedScenarioState int a;` `@ProvidedScenarioState int b;` and a step `two_numbers(int x, int y)` that stores them and returns `this` (the `GivenIngredients` pattern, BDD p.14).
- `WhenCalculator` extends `Stage<WhenCalculator>` marks `a / b` as `@ExpectedScenarioState`, calls the production calculator, and `@ProvidedScenarioState`-exports `int result` (the `WhenCook` consume-act-provide shape, BDD p.15).
- `ThenResult` extends `Stage<ThenResult>` consumes `result` via `@ExpectedScenarioState` and asserts in a void step: `assertThat(result).isEqualTo(expected);` (the `ThenMeal` pattern with AssertJ's `isEqualTo`, BDD p.16).

Running it prints the scenario back as prose — `Given two numbers 500 500 / When they are added up / Then the result is 1000` — in the console report and as a green row in the HTML5 app (BDD p.17–18). This is precisely the portfolio deliverable chain the lab prescribes: story → GWT → JGiven → AssertJ (BDDLab p.1).

Lab Deep Dive — TestLab2: Behavior Driven Testing (BDDLab p.1)

The handout's identity and structure

The lab handout is titled "[TestLab2] Behavior Driven Testing" (BDDLab p.1) — i.e. it is the *second* testing lab of the course, building on the earlier unit-testing lab. It is a single page with three parts: (1) an introduction headed "**Introduction – User Stories and BBD**" (the handout itself misspells BDD as "BBD" in this heading — reproduce the correct "BDD" in your own answers); (2) **Figure 1**, a two-column table mapping three user stories to BDD scenarios; and (3) a **Portfolio** checklist of three deliverables (BDDLab p.1).

User stories — the handout's definition and BOTH templates

The handout defines user stories precisely: "**User stories are short and simple descriptions of capabilities written from the perspective of the person who desires the new capability**" (BDDLab p.1). It then gives **two** interchangeable templates — most summaries quote only the first, but the handout explicitly lists an alternative:

1. "**As a [user type], I want [some goal] so that [some reason].**" (BDDLab p.1)
2. Alternative: "**As a [user type], I want [some goal] because [why].**" (BDDLab p.1)

Both templates carry the same three slots — *who* wants it (user type), *what* they want (goal), and *why* (reason/why) — differing only in the connective ("so that" vs "because"). The "why" slot is what distinguishes a user story from a bare feature request, and, as the Figure-1 mappings show, it is also the slot that *drops out* when the story is converted to a Given/When/Then scenario: scenarios verify the goal, not the motivation.

Figure 1 in full — the complete story→scenario table

The handout's Figure 1 ("Example of how to map User Story to BDD scenario", BDDLab p.1) contains exactly three rows:

#	User story	BDD scenario
1	"As a calculator user, I want to add two numbers so that I can do addition."	Given I have two numbers 500 & 500 / When I add them up / Then I should get result 1000
2	"As a Math teacher, I want to automate marks sorting process so that I can declare top 5 in my class."	Given a list of numbers / When I sort the list / Then the list will be in numerical order
3	"As a QA engineer, I want to check a critical feature so that I can do smoke test easily."	Given I visit Google.com / When I type 'TestingWhiz' as a search string / Then I should get search results matching TestingWhiz

Three observations the table supports: (a) every scenario instantiates the story's abstract goal with **concrete data** (two numbers → "500 & 500"; expected result → "1000") — scenarios are *examples*, not restatements; (b) the user type ("calculator user", "Math teacher", "QA engineer") frames whose behaviour is being specified but does not literally appear in the scenario clauses; (c) the third row's story names **smoke testing** (a quick check of a critical feature) as the motivation, showing BDD scenarios can serve as automated smoke tests of system-level flows (BDDLab p.1).

Portfolio deliverables — the lab's exact requirements

The Portfolio section lists three bullets (BDDLab p.1):

1. **Map your User Stories to BDD Given-When-Then Scenarios** — take the stories from your own project (written in earlier course work) and convert each, exactly as Figure 1 demonstrates.
2. **Use [JGiven] to automate your BDD Scenarios** — the scenario must become a runnable JGiven test (the deck's p.10–16 machinery), not remain prose.
3. **For domain specific assertions use the [AssertJ library]. For Swing applications use the [AssertJ-swing] to automate the Scenarios** — i.e. AssertJ inside the Then steps (with custom domain assertions per BDD p.28 where the domain warrants them), and AssertJ-Swing as the automation driver when the application under test is a Swing GUI (BDD p.29).

Note the tool chain the lab fixes: **user story** → **GWT scenario** → **JGiven** → **AssertJ** (→ **AssertJ-Swing for Swing**). The bracketed names in the handout ([JGiven], [AssertJ library], [AssertJ-swing]) are

hyperlinks to the tools' sites. Unlike some other labs in the course, this handout names **no** JHotDraw, Featureous, JRipples, Maven or GitHub steps — its scope is purely the story-to-automated-scenario pipeline; build/CI integration is covered by the deck's mention of JGiven's **Maven and Jenkins plugins** (BDD p.20).

How the lab and the deck interlock

The lab supplies the **requirements end** of the pipeline (user stories and their mapping to GWT), which the deck deliberately skips — the deck's first scenario simply *appears* as prose on p.5 without saying where scenarios come from. Conversely, the deck supplies the **automation end** (JGiven stages, state transfer, reports, AssertJ) that the lab's portfolio bullets demand but do not explain. Put together they cover the whole Verification-phase workflow: requirement (story) → acceptance criterion (scenario) → executable test (JGiven) → assertion (AssertJ/AssertJ-Swing) → evidence (console/HTML5 report) (BDDLab p.1; BDD p.5–29).

Slide-by-Slide Companion (BDD p.1–30)

This section walks the deck in order, slide by slide, with the exam angle for each. It complements the Source Map table (which gives one line per slide) with enough prose that any single slide could be answered on alone.

p.1–2 — Title and part divider

The title slide reads "**Software Verification — Pragmatic BDD for Java (SB5-MAI)**", by **Jan Corfixen Sørensen, University of Southern Denmark** (BDD p.1). Two signals in the title: "*Software Verification*" names the change-process phase being taught, and "*Pragmatic BDD for Java*" announces the deck's tool-first stance — this is BDD as you would actually practise it on a Java codebase, not BDD theory. Slide 2 is the divider opening part one, "Behavior-Driven Development" (BDD p.2). The deck has three parts marked by dividers: BDD (p.2), JGiven (p.8), AssertJ (p.21).

p.3 — Why BDD? The five typical test issues

The deck's complete list of "Typical Test issues" (BDD p.3): (1) many technical and often irrelevant details; (2) point of the test often hard to grasp; (3) code duplication; (4) can only be read by developers; (5) cannot be used as documentation. Memorise all five — they are the problem statement that every later slide answers: GWT structure answers (1) and (2); stage classes answer (3) (BDD p.11); domain language and reports answer (4) and (5) (BDD p.4, p.17–18).

p.4 — The four defining properties of BDD

Verbatim content (BDD p.4): behaviour is described in a **common domain language understandable by domain experts**; **domain experts and developers collaborate** on defining the behaviour; **executed like normal tests**; **creates a living documentation**. These four bullets are the deck's *definition* of BDD — if an exam asks "what is BDD?", this slide is the answer, with the p.3 issues as motivation.

p.5 — The pancake scenario

The deck's only plain-text scenario (BDD p.5): Scenario: a pancake can be fried out of an egg milk and flour, then Given an egg / And some milk / And the ingredient flour / When the cook mangles everything to a dough / And the cook fries the dough in a pan / Then the resulting meal is a pancake. Structural points: three Given clauses (one Given + two And), two When clauses (one When + one And), one Then clause; And continues whichever keyword preceded it. This exact scenario is then re-

expressed in Java (p.10), decomposed into stages (p.14–16), and printed back as a report (p.17) — it is the deck's spine.

p.6 — "Classical" BDD frameworks

The complete roster with their scenario formats (BDD p.6): **Cucumber** — Plain Text + Java; **JBehave** — Plain Text + Java; **Concordion** — HTML + Java; **Fitness** (i.e. Fitnesse) — Wiki + Java; **robotframework.org**. The slide's single-line verdict, set off beneath the list: "**Additional Maintenance Cost**". Know the format pairings — Concordion is the HTML one, Fitnesse the wiki one — and that the cost arises from keeping the separate scenario artefact and the Java glue in sync.

p.7 — Developer-friendly BDD frameworks

The complete roster with their languages (BDD p.7): **Spock** — Groovy; **ScalaTest** — Scala; **Jnario** — Xtend; **Serenity*** — Java (starred on the slide); **JGiven** (Java; the deck's choice). The discriminator versus p.6: the scenario *is code* in one language, no separate text file. Watch the trap: Serenity and JGiven are both Java and both developer-friendly; Spock is Groovy, not Java.

p.8–9 — JGiven divider and introduction

Slide 8 opens part two ("JGiven"). Slide 9's five-bullet introduction (BDD p.9): **developer friendly (low maintenance overhead); readable test code (Given-When-Then); modular and reusable test code; reports for domain experts; open source, <http://jgiven.org>**. Each bullet is later substantiated: low maintenance by TNG's report (p.19), readability by p.10, modularity by stages (p.11), reports by p.17–18, open source by the Apache-2 licence on p.20.

p.10 — The scenario as a Java test method

The pancake scenario as a JUnit `@Test` method named `a_pancake_can_be_fried_out_of_an_egg_milk_and_flour()`, whose body chains `given().an_egg().and().some_milk().and().the_ingredient("flour"); then when().the_cook_mangles_everything_to_a_dough().and().the_cook_fries_the_dough_in_a_pan(); then then().the_resulting_meal_is_a_pancake();` (BDD p.10). Note: the **method name itself** becomes the scenario title in the report ("A pancake can be fried out of an egg milk and flour", p.17). Everything — title, steps, arguments — lives in one Java file.

p.11 — Stage classes in general

All four bullets (BDD p.11): JGiven scenarios are **built from stage classes**; stage classes provide **modularity** and **reuse**; stage classes are a **unique feature of JGiven, not present in any other BDD framework**; typically a stage class is used for **either a Given, When, or Then** step of a scenario. The "unique feature" claim is exam bait — it is the one architectural idea JGiven adds over every other framework on p.6–7.

p.12–13 — State transfer: the diagram and the annotations

Slide 12 is purely visual: three coloured horizontal bands labelled (rotated, on the left edge) **Given Stage** (green), **When Stage** (blue), **Then Stage** (orange). In the Given band, "some state" produces `state1` and "another state" produces `state 2`; `state1` flows down into the When band where "some action" produces `result`; the Then band receives `state1`, `result`, and `state 2`, feeding "some assertion" (BDD p.12). The diagram's teaching point: state only flows **downward** through the GWT pipeline, and a Then stage may consume state originating in *any* earlier stage — including state (like `state 2`) that skipped the When stage

entirely. Slide 13 then names the mechanism: fields annotated `@ScenarioState`; **values are written and read between stages**; `@ProvidedScenarioState` and `@ExpectedScenarioState` as the (directional) alternative (BDD p.13).

p.14–16 — The three stage classes

`GivenIngredients` (p.14), `WhenCook` (p.15), `ThenMeal` (p.16) — covered line by line in the Key Concepts section above. One-slide summaries for quick recall: p.14 = provider stage, list field `@ProvidedScenarioState`, wrapper steps delegating to `the_ingredient(String)`, every step `return this`. p.15 = the four-annotation stage (`@Autowired @ScenarioState Cook`, `@ExpectedScenarioState ingredients`, `@ProvidedScenarioState dough and meal`) with AssertJ null-guards and the production call `cook.fryDoughInAPan(dough)`. p.16 = consumer stage, `@ExpectedScenarioState String meal`, `void` steps, delegation to a parameterised step holding `assertThat(meal).isEqualTo(expectedMeal)`, full imports visible including the static import of `Assertions.assertThat`.

p.17–18 — Console output and the HTML5 app

p.17 shows the runner's console rendering: `Test Class:`
`com.tngtech.jgiven.examples.pancakes.test.SpringPancakeScenarioTest`, the humanised scenario title, and the GWT prose (with the slide's own typo "mangles everthing"). The test-class name confirms the example runs under **Spring** (`SpringPancakeScenarioTest`), matching the `@Autowired` on p.15. p.18 is the HTML5 report screenshot, inventoried in full in the Key Concepts section above — headline numbers: **53 Successful, 0 Failed, 0 Pending, 53 Total (0.077s)** (BDD p.17–18).

p.19 — TNG practical experience

All seven bullets (BDD p.19): 3 years of experience in a large Java Enterprise project (up to 70 developers); over 3000 scenarios; readability and reusability of test code greatly improved; maintenance costs of automated tests reduced (**no hard numbers**); well accepted by developers; easy to learn by new developers; developers and domain experts collaborate using scenarios. The last bullet matters: even though experts cannot *write* JGiven scenarios (p.20), TNG reports experts and developers **collaborating using scenarios** — collaboration happens around the readable scenarios and reports, authorship stays with developers.

p.20 — JGiven summary

All eight bullets (BDD p.20): developer friendly; highly modular and reusable test code; just Java, no further language is needed; easy to integrate into existing test infrastructures (**JUnit, TestNG**); Open Source (**Apache 2 Licence**); **Maven and Jenkins plugins**; nice reports for domain experts; **domain experts can not write scenarios in JGiven**. Seven strengths and one limitation — and the limitation is the deck's single most exam-likely fact.

p.21–23 — AssertJ divider, motivation, and "Why AssertJ?"

Slide 21 opens part three. Slide 22's motivation, near-verbatim: "JUnit's assertions underpowered from the start, developers use frameworks like Hamcrest and Fest. Seeing a confusion of JUnit, Hamcrest and Fest." Then the three judgements: **JUnit is very simplistic; Fest looks to be abandonware; Hamcrest is stagnant and ugly** (BDD p.22). Slide 23 gives the six reasons *for* AssertJ: **still actively maintained; near complete superset of functionality; well designed; easy to get started; easy to enhance; easy to read** (BDD p.23). Pair them in answers: the p.22 complaints are the disease, the p.23 list the cure.

p.24–25 — Basic use and the worked example

p.24's four design points: `Assertions.assertThat()` **type-specific factory methods**; `AbstractAssert` **type-specific assertions**; **fluent API**; **good use of polymorphism** — plus the documentation pointer <http://joel-costigliola.github.io/assertj> (BDD p.24). p.25 demonstrates all four in one `@Test` `shouldProvideAnExample()`: the same `contains("is").startsWith("This")` chain applied to a `String` ("This is a test") and to a `String[]` ({"This", "is", "a", "test"}), with the slide's own bullets: `assertThat` factory yields `String` or `Array` `assert` as needed; `contains` and `startsWith` polymorphic; fluent style i.e. `.contains().startsWith()` (BDD p.25).

p.26–27 — Custom conditions

p.26's two claims: custom conditions **allow for more elaborate tests**, and **can be applied to types that already have a custom assertion in a straightforward way** (BDD p.26) — i.e. Conditions and custom assertions compose rather than compete. p.27's example, `shouldBeEvenlyDivisibleBySix()`: an anonymous `Condition<Integer>` overriding `public boolean matches(Integer value) to return (value % 6) == 0;`, then `assertThat(12).is(evenDivBySix);` and `assertThat(8).isNot(evenDivBySix);`. The slide's bullets: **implement a basic Condition functor**; **use your condition with is and isNot method** (BDD p.27).

p.28 — Custom assertions

The slide's own motivating prose: "Sometimes you've your own types with corresponding common assertions you'd like to apply to them. Perhaps you want to know if a `Student` instance `isInMiddleSchool?`" Then the exact three-step recipe (BDD p.28): (1) subclass `AbstractAssert` for `Student` and implement a `isInMiddleSchool`; (2) subclass `Assertions` adding a new `assertThat` factory method for your `StudentAssert` you created; (3) use your new factory and assertions as you'd always — i.e. `assertThat(student).isInMiddleSchool()`. Note both subclassings are required: the `assert` class gives you the method, the factory subclass gives you the entry point.

p.29 — AssertJ-Swing

All seven capabilities (BDD p.29): simulation of user interaction with a GUI (e.g. **drag 'n drop**); reliable GUI component lookup (**by type, by name or custom search criteria**); support for **all Swing components included in the JDK**; compact and powerful API for creation and maintenance of **functional GUI tests**; ability to **embed screenshots of failed GUI tests in HTML test reports**; can be used with **either TestNG or JUnit**; supports testing **violations of Swing's threading rules**. Seven bullets, seven possible exam fragments — the screenshot-embedding and threading-rule bullets are the two most distinctive.

p.30 — Getting started

The closing slide points to two AssertJ documents (BDD p.30): "To get quickly started, see **[The one minute starting guide]**" and "If you have a bit more time, start with **[The getting started guide]**" — onboarding pointers for the lab portfolio, no new technical content.

Compare & Contrast Tables

Classical vs developer-friendly BDD frameworks

Dimension	Classical (BDD p.6)	Developer-friendly (BDD p.7)
Members	Cucumber, JBehave, Conordion, Fitnesse, RobotFramework	Spock, ScalaTest, Jnario, Serenity, JGiven
Scenario artefact	Separate plain-text / HTML / wiki file	Ordinary code in the production language
Format per tool	Cucumber & JBehave: plain text + Java; Conordion: HTML + Java; Fitnesse: wiki + Java	Spock: Groovy; ScalaTest: Scala; Jnario: Xtend; Serenity & JGiven: Java
Glue layer	Step definitions matched to scenario lines (kept in sync by hand)	None — steps are method calls
Deck's verdict	"Additional Maintenance Cost" (BDD p.6)	Low maintenance overhead (BDD p.9)
Who can author scenarios	In principle non-developers (prose/wiki)	Developers only (BDD p.20)
Who can read the output	Anyone	Anyone — via generated reports (BDD p.17–18)

JGiven vs Cucumber — the head-to-head the deck implies

Aspect	Cucumber	JGiven
Scenario lives in	<code>.feature</code> plain-text file (Gherkin) (BDD p.6)	A Java <code>@Test</code> method (BDD p.10)
Artefacts to keep in sync	Two (feature file + step definitions)	One (BDD p.10)
IDE refactoring support	Weak across the text/code boundary	Full — it is just Java (BDD p.20)
Domain-expert authorship	Possible (plain text)	Not possible (BDD p.20)
Domain-expert readability	The feature file itself	The generated console/HTML5 reports (BDD p.17–18)
Unique structural feature	—	Stage classes (unique to JGiven, BDD p.11)
Maintenance verdict	"Additional Maintenance Cost" (BDD p.6)	Maintenance costs reduced at TNG (BDD p.19)

JUnit vs Hamcrest vs Fest vs AssertJ (the p.22–23 quadrille)

Library	Deck's judgement	Source
JUnit assertions	"Underpowered from the start"; "very simplistic"	(BDD p.22)
Fest	"Looks to be abandonware" (unmaintained)	(BDD p.22)
Hamcrest	"Stagnant and ugly"	(BDD p.22)
AssertJ	Actively maintained; near-complete superset; well designed; easy to get started / enhance / read	(BDD p.23)

The historical chain the deck sketches: JUnit's weakness pushed developers to bolt on Hamcrest and Fest, producing "a confusion of JUnit, Hamcrest and Fest" in real codebases (BDD p.22); AssertJ replaces the whole confusion with one maintained superset (BDD p.23).

Built-in assertion vs custom Condition vs custom assertion

	Built-in assertion	Custom Condition	Custom assertion
What you write	Nothing — use <code>assertThat(x).isEqualTo(...)</code> etc.	Subclass/instantiate <code>Condition<T></code> , override <code>boolean matches(T)</code>	Subclass <code>AbstractAssert</code> and subclass <code>Assertions</code> (new factory)
How you invoke it	Directly on the fluent chain	<code>.is(condition)</code> / <code>.isNot(condition)</code>	Your own factory: <code>assertThat(student).isInMiddleSchool()</code>
Granularity	Generic, type-level checks	One named reusable predicate	A whole domain type's assertion vocabulary
Example in deck	<code>isEqualTo</code> , <code>isNotNull</code> , <code>contains</code> , <code>startsWith</code> (BDD p.15–16, p.25)	<code>evenDivBySix</code> on <code>Integer</code> (BDD p.27)	<code>StudentAssert.isInMiddleSchool()</code> (BDD p.28)
Compose?	—	Can be applied to types that already have a custom assertion (BDD p.26)	Can host conditions (BDD p.26)

The three scenario-state annotations

Annotation	Direction	Meaning	Deck example
<code>@ProvidedScenarioState</code>	Out (producer)	This stage creates the value for later stages	<code>ingredients</code> in <code>GivenIngredients</code> (BDD p.14); <code>dough</code> , <code>meal</code> in <code>WhenCook</code> (BDD p.15)
<code>@ExpectedScenarioState</code>	In (consumer)	This stage requires a value provided earlier	<code>ingredients</code> in <code>WhenCook</code> (BDD p.15); <code>meal</code> in <code>ThenMeal</code> (BDD p.16)
<code>@ScenarioState</code>	Both (bidirectional)	Value is written and read between stages	<code>Cook cook</code> in <code>WhenCook</code> , paired with <code>@Autowired</code> (BDD p.13, p.15)

Mnemonic from the pitfalls section, worth repeating: **Provided = producer, Expected = consumer** (BDD p.13).

One behaviour, three renderings (the pancake's life cycle)

Rendering	Where	Form
Plain-text scenario	BDD p.5	<code>Given an egg / And some milk / ... / Then the resulting meal is a pancake</code>
Executable Java	BDD p.10	<code>given().an_egg().and().some_milk()... then().the_resulting_meal_is_a_pancake();</code>
Report prose	BDD p.17	Console prints the identical GWT text back from the executed run

The exam point of the trio: the *same words* travel from specification (p.5) to implementation (p.10) to evidence (p.17) — that round trip is what "living documentation" (BDD p.4) means operationally, and the

absence of any translation step between the three is what kills the classical frameworks' maintenance cost (BDD p.6).

The five p.3 test issues vs their BDD fixes

Test issue (BDD p.3)	BDD's fix	Source
Many technical and often irrelevant details	GWT structure surfaces only context/action/outcome; plumbing hides in stages	(BDD p.5, p.11)
Point of the test often hard to grasp	Scenario title + GWT prose state the behaviour outright	(BDD p.5, p.10)
Code duplication	Stage classes: steps written once, reused across scenarios	(BDD p.11)
Can only be read by developers	Domain language + reports for domain experts	(BDD p.4, p.9, p.17–18)
Cannot be used as documentation	Living documentation regenerated from every run	(BDD p.4, p.17–18)

Annotated API Surface — Every Identifier the Deck Shows

A semantic-search-friendly inventory of every class, annotation, method, and URL that appears in the deck's code and bullets, with role and slide.

JGiven identifiers

Identifier	Role	Slide
<code>@Test</code>	JUnit annotation marking the scenario method	(BDD p.10, p.25, p.27)
<code>given()</code> , <code>when()</code> , <code>then()</code> , <code>and()</code>	Fluent scenario keywords as Java method calls	(BDD p.10)
<code>Stage<SELF></code> (<code>com.tngtech.jgiven.Stage</code>)	Base class of every stage; self-typed generic enables fluent chaining	(BDD p.14–16)
<code>@ScenarioState</code>	Bidirectional state-sharing annotation on stage fields	(BDD p.13, p.15)
<code>@ProvidedScenarioState</code>	Producer-side state annotation	(BDD p.13, p.14, p.15)
<code>@ExpectedScenarioState</code> (<code>com.tngtech.jgiven.annotation.ExpectedScenarioState</code>)	Consumer-side state annotation	(BDD p.13, p.15, p.16)
<code>GivenIngredients</code> , <code>WhenCook</code> , <code>ThenMeal</code>	The three example stage classes	(BDD p.14–16)
<code>an_egg()</code> , <code>some_milk()</code> , <code>the_ingredient(String)</code>	Given steps; the first two delegate to the third	(BDD p.14)
<code>the_cook_mangles_everything_to_a_dough()</code> , <code>the_cook_fries_the_dough_in_a_pan()</code>	When steps (only the second's body is shown)	(BDD p.10, p.15)
<code>the_resulting_meal_is_a_pan_cake()</code> , <code>the_resulting_meal_is_a(String)</code>	Then steps; the first delegates to the second	(BDD p.16)

Identifier	Role	Slide
<code>com.tngtech.jgiven.examples.pancakes.test.steps</code>	Package of the example (TNG's own repo)	(BDD p.16)
<code>SpringPanCakeScenarioTest</code> (<code>com.tngtech.jgiven.examples.pancakes.test</code>)	The test class shown running in the console	(BDD p.17)
http://jgiven.org	JGiven project site	(BDD p.9)

Spring and example-domain identifiers

Identifier	Role	Slide
<code>@Autowired</code>	Spring DI annotation injecting the <code>Cook</code> into the stage	(BDD p.15)
<code>Cook / cook.fryDoughInAPan(dough)</code>	The production collaborator and the action under test	(BDD p.15)
<code>List<String> ingredients, Set<String> dough, String meal</code>	The scenario-state payloads flowing Given→When→Then	(BDD p.14–16)

AssertJ identifiers

Identifier	Role	Slide
<code>org.assertj.core.api.Assertions.assertThat</code> (static import)	Entry point of every fluent assertion	(BDD p.16, p.24)
<code>assertThat(actual)</code>	Type-specific factory — yields String-assert, Array-assert, etc.	(BDD p.24–25)
<code>AbstractAssert</code>	Base class of all type-specific assertions; subclass it for custom assertions	(BDD p.24, p.28)
<code>Assertions</code>	Factory-holder class; subclass it to add your own <code>assertThat</code>	(BDD p.28)
<code>contains(...), startsWith(...)</code>	Polymorphic fluent checks (String and array)	(BDD p.25)
<code>isEqualTo(...), isNotNull()</code>	Equality and null-guard assertions used in the stages	(BDD p.15–16)
<code>Condition<T> / boolean matches(T value)</code>	Reusable predicate functor — override <code>matches</code>	(BDD p.27)
<code>is(condition), isNot(condition)</code>	Apply a condition positively / negatively	(BDD p.27)
<code>StudentAssert, isInMiddleSchool()</code>	The custom-assertion example for <code>Student</code>	(BDD p.28)
<code>evenDivBySix</code>	The example <code>Condition<Integer> (value % 6 == 0)</code>	(BDD p.27)
<code>shouldProvideAnExample(), shouldBeEvenlyDivisibleBySix()</code>	The two example test methods	(BDD p.25, p.27)
http://joel-costigliola.github.io/assertj	AssertJ documentation site	(BDD p.24)

Definitions & Terminology

Term	Definition	Source
Behaviour-Driven Development (BDD)	<i>What:</i> a practice for specifying and verifying software by its externally observable behaviour, described in a common domain language understandable by domain experts; experts + developers collaborate to define it; it is executed like normal tests and forms living documentation. <i>Used for:</i> making Verification-phase acceptance criteria simultaneously readable (by experts), runnable (by developers) and self-documenting, fixing the "developer-only, undocumented, cluttered" failures of ordinary tests. <i>Applied:</i> capture behaviour as Given/When/Then before coding, automate in JGiven, assert in AssertJ, run in CI.	(BDD p.4)
Living documentation	<i>What:</i> documentation that is the executing test itself, regenerated from each run, so it cannot go stale — a failing executable spec signals the drift. <i>Used for:</i> giving experts a trustworthy, always-current description of behaviour without a separate doc to maintain. <i>Applied:</i> the JGiven console + HTML5 reports render the run scenario back as prose.	(BDD p.4, p.17–18)
Given / When / Then (GWT)	<i>What:</i> the three-part scenario structure — pre-conditions (Given), action/event (When), expected outcome/assertion (Then); extra clauses chained with And . <i>Used for:</i> forcing every test to state its assumptions, trigger, and expected result so its purpose is obvious to any reader. <i>Applied:</i> Given an egg ... When the cook fries the dough ... Then the meal is a pancake .	(BDD p.5)
Scenario	<i>What:</i> a single concrete behaviour written in Given/When/Then; in JGiven, exactly one Java test method. <i>Used for:</i> being the unit of acceptance criterion the Verification phase checks. <i>Applied:</i> one scenario per user story (BDDLab p.1).	(BDD p.5, p.10)
Gherkin	<i>What:</i> the plain-text Given/When/Then language used by classical frameworks like Cucumber (.feature files). <i>Used for:</i> letting non-developers author scenarios as prose, at the cost of separate step-definition glue. <i>Applied:</i> a .feature file matched to Java steps by regex.	(BDD p.6)
Ubiquitous / domain language	<i>What:</i> the shared vocabulary of domain experts and developers in which scenarios (and ideally assertions) are written. <i>Used for:</i> eliminating translation errors between requirements and tests so experts can read what is verified. <i>Applied:</i> domain-named steps (the_cook_fries_the_dough_in_a_pan) and domain assertions (assertThat(student).isInMiddleSchool()).	(BDD p.4); [Nor06]
Executable specification	<i>What:</i> a specification of behaviour that also runs as a test, so document and test are one file. <i>Used for:</i> preventing spec/code drift — a stale spec fails when run. <i>Applied:</i> a JGiven scenario whose prose report is produced from its own execution.	(BDD p.4)
Classical BDD framework	<i>What:</i> scenario kept in a separate plain-text/markup file glued to Java step definitions — Cucumber, JBehave, Concordion, Fitnesse, RobotFramework. <i>Used for:</i> enabling non-developer authorship, but carrying an "additional maintenance cost" (two artefacts, regex glue to keep in sync). <i>Applied:</i> write a .feature file + matching step methods.	(BDD p.6)
Developer-friendly BDD framework	<i>What:</i> scenario written in the production language, no plain-text glue — Spock, ScalaTest, Jnario, Serenity, JGiven. <i>Used for:</i> lowering maintenance and gaining IDE support, at the cost of non-developers no longer being able to author scenarios. <i>Applied:</i> the scenario is just code in the project's own language.	(BDD p.7)
JGiven	<i>What:</i> open-source (Apache 2) developer-friendly BDD framework for Java; scenarios are JUnit/TestNG methods built from stage classes; generates console + HTML5 reports. <i>Used for:</i> turning agreed GWT scenarios into runnable, CI-able, expert-readable acceptance tests with no .feature file. <i>Applied:</i> given().an_egg().and()... reading as Java.	(BDD p.9, p.20)

Term	Definition	Source
Stage class	<i>What:</i> a JGiven class (<code>extends Stage<SELF></code>) holding the step methods for (typically) one of Given, When, or Then; unique to JGiven. <i>Used for:</i> modularity and reuse — a step is written once and reused across scenarios, attacking test duplication. <i>Applied:</i> <code>GivenIngredients</code> , <code>WhenCook</code> , <code>ThenMeal</code> , each chaining via <code>return this</code> .	(BDD p.11)
@ScenarioState	<i>What:</i> annotation on a stage field whose value is transferred (both read and written — bidirectional) between stages. <i>Used for:</i> sharing a value that a stage both consumes and exposes (e.g. the injected <code>Cook</code>). <i>Applied:</i> <code>@ScenarioState Cook cook;</code> in <code>WhenCook</code> .	(BDD p.13, p.15)
@ProvidedScenarioState	<i>What:</i> marks a stage field this stage produces/provides for later stages (<i>Provided</i> = <i>producer</i>). <i>Used for:</i> exporting state down the Given→When→Then pipeline without parameter passing. <i>Applied:</i> <code>@ProvidedScenarioState List<String> ingredients</code> in <code>GivenIngredients</code> .	(BDD p.13, p.14)
@ExpectedScenarioState	<i>What:</i> marks a stage field this stage consumes/expects , provided by an earlier stage (<i>Expected</i> = <i>consumer</i>). <i>Used for:</i> importing upstream state so a later stage can act on or assert it. <i>Applied:</i> <code>@ExpectedScenarioState String meal</code> in <code>ThenMeal</code> .	(BDD p.13, p.15–16)
@Autowired	<i>What:</i> Spring dependency-injection annotation; JGiven integrates with Spring. <i>Used for:</i> having the container supply a stage's collaborator rather than the test constructing it. <i>Applied:</i> <code>@Autowired @ScenarioState Cook cook;</code> injects the <code>Cook</code> .	(BDD p.15)
AssertJ	<i>What:</i> actively-maintained, fluent, polymorphic Java assertion library; near-complete superset of JUnit/Hamcrest/Fest. <i>Used for:</i> replacing the "JUnit + Hamcrest + Fest confusion" with one readable assertion API used in Then steps. <i>Applied:</i> <code>assertThat(actual).contains("is").startsWith("This")</code> .	(BDD p.22–24)
assertThat()	<i>What:</i> AssertJ's type-specific factory method that returns the right assertion object for the value's type (overloaded/polymorphic). <i>Used for:</i> entering the fluent chain with the correct assert for Strings, arrays, numbers, domain types, etc. <i>Applied:</i> <code>assertThat(actualArray)</code> yields an array-assert, <code>assertThat("x")</code> a String-assert.	(BDD p.24)
Fluent API	<i>What:</i> chained, sentence-like method calls returning the assertion object each time. <i>Used for:</i> making assertions read like prose so the Then step doubles as documentation. <i>Applied:</i> <code>assertThat(x).contains("is").startsWith("This")</code> .	(BDD p.24–25)
AbstractAssert	<i>What:</i> AssertJ base class for all type-specific assertions. <i>Used for:</i> being the class you subclass to add domain-specific assertions. <i>Applied:</i> <code>class StudentAssert extends AbstractAssert<...></code> implementing <code>isInMiddleSchool()</code> .	(BDD p.24, p.28)
Condition (functor)	<i>What:</i> a reusable AssertJ predicate object (override <code>matches</code>) used via <code>.is(...)</code> / <code>.isNot(...)</code> . <i>Used for:</i> packaging bespoke pass/fail logic once and reusing it by name across assertions. <i>Applied:</i> <code>Condition<Integer> evenDivBySix ... assertThat(12).is(evenDivBySix)</code> .	(BDD p.26–27)
Custom assertion	<i>What:</i> a domain-specific fluent assertion built by subclassing <code>AbstractAssert</code> + <code>Assertions</code> . <i>Used for:</i> pushing domain language into the assertion layer so the Then step speaks the domain. <i>Applied:</i> <code>assertThat(student).isInMiddleSchool()</code> .	(BDD p.28)
AssertJ-Swing	<i>What:</i> AssertJ module for automating Swing GUI tests: simulate user actions (drag 'n drop), reliable component lookup, screenshots on failure, EDT threading-rule checks. <i>Used for:</i> extending Verification to the GUI layer of Swing apps, catching interaction and threading bugs invisible to plain assertions. <i>Applied:</i> drive a JHotDraw drag-and-drop and assert the canvas state.	(BDD p.29)
User story	<i>What:</i> a short capability description from the user's perspective: "As a [user type], I want [goal] so that [reason]." <i>Used for:</i> being the requirement that BDD turns into a Given/When/Then acceptance criterion. <i>Applied:</i> "As a calculator user, I want to add two numbers so that I can do addition" → a GWT scenario.	(BDDLab p.1)

Term	Definition	Source
Verification (phase)	<i>What:</i> the change-process phase confirming the change is correct/complete with tests passing, acceptance criteria (BDD scenarios) met, and no regressions. <i>Used for:</i> gating a change as "done" only when its scenarios are green. <i>Applied:</i> the change is verified when the JGiven HTML report shows 0 Failed / 0 Pending.	[Raj 13] ; course overview
Step method	<i>What:</i> a public method on a stage class whose snake_case name becomes one line of scenario prose; Given/When steps <code>return this</code> (the stage) for chaining, Then steps may be <code>void</code> . <i>Used for:</i> expressing one GWT clause as executable Java whose name doubles as report text. <i>Applied:</i> <code>the_cook_fries_the_dough_in_a_pan()</code> prints as "the cook fries the dough in a pan".	(BDD p.10, p.14–17)
Step delegation (intra-stage reuse)	<i>What:</i> a step method calling another step method of the same stage, so one parameterised step is the single point of truth and named wrappers add readability. <i>Used for:</i> reuse inside a stage, complementing stage-level reuse across scenarios. <i>Applied:</i> <code>an_egg()</code> → <code>the_ingredient("egg")</code> ; <code>the_resulting_meal_is_a_pan_cake()</code> → <code>the_resulting_meal_is_a("pancake")</code> .	(BDD p.14, p.16)
Self-type generic (Stage<SELF>)	<i>What:</i> the idiom of a stage extending <code>Stage</code> parameterised by itself (<code>class WhenCook extends Stage<WhenCook></code>). <i>Used for:</i> keeping the fluent chain statically typed as the concrete stage so <code>return this</code> exposes that stage's step vocabulary. <i>Applied:</i> all three example stages declare themselves this way.	(BDD p.14–16)
Console report	<i>What:</i> JGiven's plain-text rendering of an executed scenario — test class name, humanised scenario title, GWT prose. <i>Used for:</i> immediate, human-readable run evidence in the terminal/CI log. <i>Applied:</i> the pancake run under <code>SpringPanCakeScenarioTest</code> .	(BDD p.17)
HTML5 app report	<i>What:</i> JGiven's browsable web report with summary counts, tag/class navigation, search, Group By/Sort By/Status/Tags/Classes controls, data tables, attachments, durations. <i>Used for:</i> the Verification dashboard domain experts can navigate without reading Java. <i>Applied:</i> the 53-scenario screenshot — "53 Successful, 0 Failed, 0 Pending, 53 Total (0.077s)".	(BDD p.18)
Pending (scenario status)	<i>What:</i> the third report status besides Successful and Failed — a declared scenario not (yet) fully executed/implemented. <i>Used for:</i> tracking acceptance criteria written ahead of the code, per BDD's behaviour-first workflow. <i>Applied:</i> the report sidebar counts Pending Scenarios separately (0 in the screenshot).	(BDD p.18)
Tag (JGiven report)	<i>What:</i> a clickable label attached to scenarios in the HTML5 report, with its own tag page. <i>Used for:</i> navigating verification evidence by feature/issue/area rather than by class. <i>Applied:</i> tag chips like <i>TestNG</i> , <i>JUnit</i> , <i>AsciiDoc Report</i> , <i>German Scenarios</i> , <i>Case Diffs</i> , <i>HTML5 Report</i> , <i>Attachments</i> in the screenshot; "Clicking on tag labels opens the tag page" is itself a listed scenario.	(BDD p.18)
TNG Technology Consulting	<i>What:</i> the company behind JGiven and the source of the deck's industrial evidence (and of the pancake example — package <code>com.tngtech...</code>). <i>Used for:</i> grounding JGiven's maintainability claims in 3 years / ≤70 developers / 3000+ scenarios of practice. <i>Applied:</i> cite for "readability and reusability greatly improved, maintenance costs reduced (no hard numbers)".	(BDD p.16, p.19)
Maven / Jenkins plugins (JGiven)	<i>What:</i> JGiven's build- and CI-integration plugins. <i>Used for:</i> running scenarios and generating reports automatically on every build, wiring Verification into continuous integration. <i>Applied:</i> listed among JGiven's strengths on the summary slide.	(BDD p.20)

Term	Definition	Source
Functor (Condition)	<i>What:</i> the deck's term for the <code>Condition</code> object — an object that exists to carry one function (<code>matches</code>). <i>Used for:</i> naming and reusing a predicate as a value passed to <code>is / isNot</code> . <i>Applied:</i> "Implement a basic Condition functor" — <code>evenDivBySix</code> .	(BDD p.27)
Event Dispatcher Thread (EDT)	<i>What:</i> Swing's single GUI thread; Swing's threading rules require GUI mutations to happen on it. <i>Used for:</i> the bug class <code>AssertJ-Swing</code> can test for — "violations of Swing's threading rules". <i>Applied:</i> relevant to any Swing app under GUI verification (course frame: <code>JHotDraw</code>).	(BDD p.29)
Smoke test	<i>What:</i> a quick automated check of a critical feature to confirm the system basically works. <i>Used for:</i> the motivation in the lab's third user story ("so that I can do smoke test easily"). <i>Applied:</i> the QA-engineer scenario — visit <code>Google.com</code> , search 'TestingWhiz', expect matching results.	(BDDL ab p.1)
TestLab 2	<i>What:</i> the lab handout's identifier — "[TestLab2] Behavior Driven Testing", the course's second testing lab. <i>Used for:</i> the portfolio work mapping user stories to GWT scenarios and automating them. <i>Applied:</i> three deliverables — map stories to scenarios, automate with <code>JGiven</code> , assert with <code>AssertJ</code> (<code>AssertJ-Swing</code> for Swing).	(BDDL ab p.1)

Common Pitfalls / Gotchas

- **"Domain experts can write JGiven scenarios" — FALSE.** The deck explicitly states domain experts **cannot** write scenarios in `JGiven` (BDD p.20); they can only *read* the generated reports. This is `JGiven`'s central trade-off versus plain-text Cucumber, where a non-developer could author the `.feature` file. Classic exam trap.
- **Confusing the two directional state annotations.** `@ProvidedScenarioState` = this stage **provides/outputs** the value; `@ExpectedScenarioState` = this stage **expects/consumes** it. `@ScenarioState` is bidirectional. Mnemonic: *Provided* = *producer*, *Expected* = *consumer* (BDD p.13).
- **Thinking classical frameworks are "better because plain text."** The deck's verdict on the plain-text class (Cucumber/JBehave/etc.) is **"Additional Maintenance Cost"** (BDD p.6) — the separate scenario file + step-definition glue must be kept in sync. `JGiven`'s whole point is to remove that by living in Java (BDD p.7, p.20).
- **Stage classes are JGiven-only.** Don't claim Cucumber or JBehave have "stage classes" — the deck says stage classes are a **unique feature of JGiven** (BDD p.11).
- **BDD ≠ a tool.** BDD is the *practice* (behaviour in domain language, GWT, collaboration, living docs, BDD p.4); `JGiven` and `AssertJ` are *tools* that implement it. Don't equate "BDD" with "Cucumber" or "JGiven."
- **AssertJ vs JUnit/Hamcrest/Fest.** The deck recommends `AssertJ` specifically because **Fest is abandonware** and **Hamcrest is stagnant/ugly**, while `AssertJ` is maintained, a near-superset, and readable (BDD p.22–23). Don't cite `Hamcrest` as the deck's choice.
- **Forgetting return this.** `JGiven`'s fluent chaining (`given().an_egg().and(). . .`) only works because every step method returns the stage (`return this;` , enabled by the self-type generic `Stage<SELF>`) (BDD p.14). A `void` step breaks the chain (Then steps may be `void` , as in `ThenMeal` , because nothing chains after them — BDD p.16).
- **given/when/then are not Gherkin keywords here — they're Java methods.** In `JGiven` the GWT structure is expressed by method calls `given()/when()/then()/and()` plus snake_case step names (BDD p.10), not a parsed plain-text file.

- **AssertJ contains / startsWith are polymorphic.** The same method names work on both `String` and `String[]` because `assertThat` returns the appropriate type-specific assert (BDD p.25). Don't assume separate APIs.
- **AssertJ-Swing's EDT point is easy to miss.** It can test **violations of Swing's threading rules** (BDD p.29) — i.e. that GUI mutations happen on the Event Dispatch Thread. Relevant exam detail for GUI verification.
- **The deck does not name JHotDraw or the eight phases.** If asked "what does the BDD deck say about JHotDraw / the change process?", the honest answer is *the deck doesn't mention them directly*; the connection is course-frame (this guide, [Raj13]). Don't fabricate a deck citation for it.
- **"Pancake" / "pan cake" / "pancake" spelling drift.** The slides use both `pancake` (method names, BDD p.10) and `pan cake` (prose scenario, BDD p.5, p.17). Not a bug — just reproduce whichever the question shows.
- **Step-name drift between p.10 and p.16.** The scenario method on p.10 calls `then().the_resulting_meal_is_a_pancake()` while the `ThenMeal` stage on p.16 defines `the_resulting_meal_is_a_pan_cake()`. The two slides are slightly inconsistent with each other — if asked to reproduce, match the slide you are quoting and don't let the mismatch throw you.
- **Serenity is Java and developer-friendly, not classical.** On p.7 Serenity is listed (starred) among the developer-friendly frameworks in *Java*. A tempting wrong answer is to lump it with Cucumber because both target acceptance testing — the deck's taxonomy puts Serenity beside JGiven (BDD p.7).
- **Condition is a predicate, not an assertion.** A `Condition<T>` overrides `boolean matches(T)` and is *applied* via `.is()` / `.isNot()` (BDD p.27); a custom assertion subclasses `AbstractAssert` and adds its own methods plus a new `assertThat` factory via an `Assertions` subclass (BDD p.28). Don't swap the recipes.
- **The custom-assertion recipe has TWO subclassing steps.** Students often remember "subclass `AbstractAssert`" and forget step 2: **also subclass `Assertions`** to add the `assertThat(Student)` factory — without it there is no entry point to your `StudentAssert` (BDD p.28).
- **JGiven report status is three-valued.** Successful / Failed / **Pending** (BDD p.18) — "pass or fail" is an incomplete description of the report model.
- **AssertJ-Swing pairs with both runners.** It "can be used with either TestNG or JUnit" (BDD p.29) — same dual-runner support as JGiven itself (JUnit, TestNG; BDD p.20). Don't claim either tool is JUnit-only.
- **The lab has TWO user-story templates.** "...so that [some reason]" **and** the alternative "...because [why]" (BDDLab p.1). Quoting only the first is incomplete if the question asks for the handout's templates.
- **The user story's "why" does not survive into the scenario.** In all three Figure-1 mappings the motivation clause ("so that I can ...") vanishes from the GWT scenario — the scenario verifies the goal with concrete example data, not the reason (BDDLab p.1).

Source-text quirks worth knowing (so they don't confuse you)

The source materials contain a handful of literal typos/quirks; knowing them prevents second-guessing during revision:

- The lab handout's introduction heading reads **"User Stories and BBD"** — "BBD" is a typo for BDD (BDDLab p.1).
- The console output on slide 17 prints **"mangles everthing"** — missing the "y" of "everything"; the method name on p.10 spells it correctly (`the_cook_mangles_everything_to_a_dough`).
- Slide 6 lists **"Fitness"** — the framework's actual name is **Fitnessse** (wiki + Java).

- Slide 28's prose reads "Sometimes **you've** your own types..." and "if **aStudent** instance" — slide-deck shorthand/typo, the meaning is "you have your own types" / "a **Student** instance" (BDD p.28).
- Slide 16 shows `the_resulting_meal_is_a( expectedMeal: "pancake" )` — the `expectedMeal:` label is an IDE parameter-name hint, not Java syntax (see the line-by-line commentary above).
- Slide 7 stars Serenity ("**Serenity***)" without expanding the footnote on the slide itself.

Exam Focus

Most likely to be asked:

1. **Define BDD and give its four properties** — domain language understandable by experts, expert+developer collaboration, executed like normal tests, living documentation (BDD p.4). Be ready to also list the *test issues* BDD fixes (BDD p.3).
2. **Write/read a Given-When-Then scenario** — explain each keyword (pre-condition / action / expected outcome) using the pancake example (BDD p.5) or a fresh one.
3. **Where does BDD fit in the change process?** — **Verification** phase; BDD scenarios are the **executable acceptance criteria** confirming correctness with no regressions [Raj13]. Tie to CI (JGiven Jenkins/Maven plugins run them automatically, BDD p.20).
4. **Classical vs developer-friendly frameworks** — name examples of each and state the discriminator: classical (Cucumber/JBehave/Concordion/Fitnesse/RobotFramework) carry "Additional Maintenance Cost" (separate plain-text + glue); developer-friendly (Spock/ScalaTest/Jnario/Serenity/JGiven) live in the production language (BDD p.6–7).
5. **JGiven mechanics** — scenario = Java test method (BDD p.10); built from **stage classes** for modularity/reuse, one per G/W/T, unique to JGiven (BDD p.11); **state transfer** via `@ScenarioState` / `@ProvidedScenarioState` / `@ExpectedScenarioState` (BDD p.13). Know the Given/When/Then stage code (BDD p.14–16).
6. **JGiven's key limitation** — domain experts **cannot author** scenarios (BDD p.20). High-value contrast question.
7. **AssertJ** — why over JUnit/Hamcrest/Fest (BDD p.22–23); the fluent `assertThat()` factory + polymorphic chained assertions (BDD p.24–25); custom `Condition` with `is` / `isNot` (BDD p.26–27); custom domain assertions by subclassing `AbstractAssert` + `Assertions` (BDD p.28).
8. **AssertJ-Swing** — for GUI verification: drag 'n drop simulation, reliable component lookup, screenshots on failure, EDT threading-rule checks (BDD p.29). Connect to JHotDraw (Swing editor).
9. **Lab task** — map a **user story** ("As a ... I want ... so that ...") to a Given/When/Then scenario, automate with JGiven + AssertJ (+ AssertJ-Swing for Swing) (BDDLab p.1). Be ready to do this on the spot with the calculator/Math-teacher/QA-engineer examples from the lab figure.
10. **Living documentation** — explain how the JGiven console + HTML5 reports (BDD p.17–18) make the executed scenario double as readable documentation, satisfying the "readable by domain experts / usable as documentation" gaps of ordinary tests (BDD p.3–4).
11. **The TNG evidence** — be able to quote the industrial figures exactly: 3 years, large Java Enterprise project, up to 70 developers, over 3000 scenarios, readability/reusability greatly improved, maintenance cost reduced *with no hard numbers*, well accepted, easy to learn, experts and developers collaborate using scenarios (BDD p.19). The "no hard numbers" caveat earns balance marks.
12. **HTML5 report literacy** — name the report's parts: SUMMARY (All/Failed/Pending), TAGS, CLASSES sidebar; Group By / Sort By / Status / Tags / Classes controls; per-scenario tags and durations; data tables, attachments, German scenarios, AsciiDoc output among the visible capabilities (BDD p.18).

13. **The custom-assertion three-step recipe verbatim** — (1) subclass `AbstractAssert` for `Student` implementing `isInMiddleSchool`; (2) subclass `Assertions` adding the new `assertThat` factory for your `StudentAssert`; (3) use as always: `assertThat(student).isInMiddleSchool()` (BDD p.28). Both subclassings, in that order.
14. **Both user-story templates and the three Figure-1 rows** — "As a [user type], I want [some goal] so that [some reason]" / alternative "...because [why]"; calculator → 500 & 500 → 1000; Math teacher → sort list → numerical order; QA engineer → Google.com + 'TestingWhiz' → matching results (BDDLab p.1).

One-line answers to keep ready: - *What is BDD?* Behaviour described in shared domain language, defined collaboratively, executed as tests, producing living documentation (BDD p.4). - *GWT?* Given = context, When = action, Then = expected outcome (BDD p.5). - *Stage classes?* JGiven's reusable per-step Java classes; unique to JGiven (BDD p.11). - *State transfer?* `@Provided` (out) / `@Expected` (in) / `@ScenarioState` (both) (BDD p.13). - *JGiven's catch?* Domain experts can read but not write scenarios (BDD p.20). - *AssertJ?* Fluent, polymorphic, maintained assertion library for the Then step (BDD p.23–25). - *Why not Hamcrest/Fest?* Fest is abandonware, Hamcrest stagnant and ugly; JUnit alone is very simplistic (BDD p.22). - *Custom assertion?* Subclass `AbstractAssert` + subclass `Assertions` for the factory; then `assertThat(student).isInMiddleSchool()` (BDD p.28). - *Condition?* A functor overriding `matches`, used with `is` / `isNot` (BDD p.27). - *AssertJ-Swing?* GUI simulation (drag 'n drop), reliable lookup, all JDK Swing components, screenshots of failures in HTML reports, TestNG or JUnit, EDT-violation testing (BDD p.29). - *TNG numbers?* 3 years, ≤70 devs, 3000+ scenarios; maintenance down (no hard numbers) (BDD p.19). - *User story template?* "As a [user type], I want [some goal] so that [some reason]" — alternative: "...because [why]" (BDDLab p.1). - *Classical frameworks?* Cucumber, JBehave (plain text), Concordion (HTML), Fitnesse (wiki), RobotFramework — additional maintenance cost (BDD p.6). - *Developer-friendly frameworks?* Spock (Groovy), ScalaTest (Scala), Jnario (Xtend), Serenity (Java), JGiven (Java) (BDD p.7). - *JGiven report statuses?* Successful, Failed, Pending — the screenshot reads "53 Successful, 0 Failed, 0 Pending, 53 Total" (BDD p.18). - *JGiven licence and CI hooks?* Apache 2 licence; Maven and Jenkins plugins; integrates with JUnit and TestNG (BDD p.20). - *Lab deliverable chain?* User story → Given-When-Then scenario → JGiven automation → AssertJ assertions (AssertJ-Swing for Swing apps) (BDDLab p.1).

Self-Test Questions with Answers

A quick-fire drill set; every answer is grounded in a specific slide. Cover the answer column, answer aloud, check.

Definitions and motivation

Q1. List all five "typical test issues" that motivate BDD. A: Many technical and often irrelevant details; point of the test often hard to grasp; code duplication; can only be read by developers; cannot be used as documentation (BDD p.3).

Q2. Give the deck's four-part characterisation of BDD. A: Behaviour described in a common domain language understandable by domain experts; domain experts and developers collaborate on defining the behaviour; executed like normal tests; creates a living documentation (BDD p.4).

Q3. In the pancake scenario, how many Given, When, and Then clauses are there, and how are continuations written? A: Three Given clauses (Given + two And), two When clauses (When + And), one Then clause; `And` continues the preceding keyword (BDD p.5).

Q4. What does each GWT keyword denote? A: Given = pre-conditions/starting context; When = the action/event under test; Then = the expected outcome/assertion (BDD p.5).

Frameworks

Q5. Name the five classical BDD frameworks and each one's scenario format. A: Cucumber (plain text + Java), JBehave (plain text + Java), Concordion (HTML + Java), Fitnesse (wiki + Java), RobotFramework — collectively: "Additional Maintenance Cost" (BDD p.6).

Q6. Name the five developer-friendly frameworks and their languages. A: Spock — Groovy; ScalaTest — Scala; Jnario — Xtend; Serenity — Java (starred); JGiven — Java (BDD p.7).

Q7. Which two frameworks on the developer-friendly slide are Java? A: Serenity and JGiven (BDD p.7).

JGiven mechanics

Q8. What five selling points does the JGiven introduction slide list? A: Developer friendly (low maintenance overhead); readable test code (Given-When-Then); modular and reusable test code; reports for domain experts; open source — <http://jgiven.org> (BDD p.9).

Q9. What is a stage class, and what is unique about it? A: The building block JGiven scenarios are built from, providing modularity and reuse; typically one per Given/When/Then; stage classes are a unique feature of JGiven, not present in any other BDD framework (BDD p.11).

Q10. Name the three state-transfer annotations and their directions. A: `@ProvidedScenarioState` — producer/out; `@ExpectedScenarioState` — consumer/in; `@ScenarioState` — bidirectional. Values are written and read between stages (BDD p.13).

Q11. In `WhenCook`, which fields carry which annotations? A: `Cook cook` — `@Autowired` `@ScenarioState`; `List<String> ingredients` — `@ExpectedScenarioState`; `Set<String> dough` and `String meal` — `@ProvidedScenarioState` (BDD p.15).

Q12. Why do Given/When step methods return this while Then steps may be void? A: `return this` (typed via the `Stage<SELF>` self-type) enables fluent chaining with `and()`; nothing chains after a terminal assertion, so Then steps can be `void` (BDD p.14–16).

Q13. How does JGiven turn code into report prose? A: `Snake_case` method names become space-separated prose and method arguments are woven in — `the_ingredient("flour")` prints as "And the ingredient flour"; the test-method name becomes the scenario title (BDD p.10, p.17).

Q14. What are JGiven's two report forms and the three scenario statuses? A: Console report and HTML5 app report; statuses Successful / Failed / Pending (BDD p.17–18).

Q15. Quote three facts from the TNG experience report. A: Any three of: 3 years on a large Java Enterprise project (up to 70 developers); over 3000 scenarios; readability/reusability greatly improved; maintenance costs reduced (no hard numbers); well accepted by developers; easy to learn; developers and domain experts collaborate using scenarios (BDD p.19).

Q16. State JGiven's eight summary bullets — and which one is a limitation. A: Developer friendly; highly modular and reusable test code; just Java, no further language needed; easy to integrate into existing test infrastructures (JUnit, TestNG); open source (Apache 2 licence); Maven and Jenkins plugins; nice reports for domain experts; **limitation: domain experts can not write scenarios in JGiven** (BDD p.20).

AssertJ

Q17. Why does the deck reject JUnit's assertions, Fest, and Hamcrest? A: JUnit is very simplistic (underpowered from the start); Fest looks to be abandonware; Hamcrest is stagnant and ugly — and mixing them produces "a confusion of JUnit, Hamcrest and Fest" (BDD p.22).

Q18. Give the six "Why AssertJ?" reasons. A: Still actively maintained; near complete superset of functionality; well designed; easy to get started; easy to enhance; easy to read (BDD p.23).

Q19. What four design elements does the "Basic Use" slide name? A: `Assertions.assertThat()` type-specific factory methods; `AbstractAssert` type-specific assertions; fluent API; good use of polymorphism (BDD p.24).

Q20. In the basic-use example, what demonstrates polymorphism? A: The same `contains("is").startsWith("This")` chain works on both a `String` and a `String[]` because the `assertThat` factory yields a `String-` or `Array-`assert as needed (BDD p.25).

Q21. Write the `evenDivBySix` condition and its two uses. A: `Condition<Integer>` overriding `matches(Integer value)` returning `(value % 6) == 0`; then `assertThat(12).is(evenDivBySix)`; and `assertThat(8).isNot(evenDivBySix)`; (BDD p.27).

Q22. Recite the custom-assertion recipe. A: (1) Subclass `AbstractAssert` for `Student` and implement `isInMiddleSchool`; (2) subclass `Assertions` adding a new `assertThat` factory for the `StudentAssert`; (3) use as always: `assertThat(student).isInMiddleSchool()` (BDD p.28).

Q23. List as many AssertJ-Swing capabilities as you can (there are seven). A: Simulation of user interaction (e.g. drag 'n drop); reliable component lookup (by type, name, or custom criteria); support for all JDK Swing components; compact, powerful API for functional GUI tests; screenshots of failed GUI tests embedded in HTML reports; works with TestNG or JUnit; tests violations of Swing's threading rules (BDD p.29).

Lab

Q24. Define a user story and give both handout templates. A: A short, simple description of a capability written from the perspective of the person who desires it; "As a [user type], I want [some goal] so that [some reason]" — alternative "As a [user type], I want [some goal] because [why]" (BDDLab p.1).

Q25. Reproduce the Math-teacher mapping from Figure 1. A: Story: "As a Math teacher, I want to automate marks sorting process so that I can declare top 5 in my class." Scenario: Given a list of numbers / When I sort the list / Then the list will be in numerical order (BDDLab p.1).

Q26. What are the lab's three portfolio deliverables? A: Map your user stories to BDD Given-When-Then scenarios; use JGiven to automate them; use AssertJ for domain-specific assertions — and AssertJ-Swing to automate scenarios for Swing applications (BDDLab p.1).

Transfer / synthesis

Q27. Where does BDD sit in the eight-phase change process, and what is the pass condition? A: Verification — the scenarios are the executable acceptance criteria; the phase passes when all scenarios are green (HTML report: 0 Failed, 0 Pending) [Raj13] (course-frame synthesis; report mechanics BDD p.18).

Q28. Why is JGiven's "just Java" property a maintenance argument and not merely a convenience? A: One artefact instead of two means no plain-text/step-definition pair to keep in sync (the

classical frameworks' "Additional Maintenance Cost", BDD p.6), full IDE refactoring support, and the TNG-reported reduction in test-maintenance cost (BDD p.19–20) — cheap-to-maintain verification is verification that keeps running over a system's life.

Source Map

Pages	Deck section	Content
BDD p.1	Title	"Software Verification — Pragmatic BDD for Java (SB5-MAI)", Jan Corfixen Sørensen, USD.
BDD p.2	Section divider	"Behavior-Driven Development".
BDD p.3	Why BDD?	Typical test issues: irrelevant details, point hard to grasp, duplication, developer-only readable, unusable as documentation.
BDD p.4	BDD defined	Domain language for experts; expert+developer collaboration; executed like tests; living documentation.
BDD p.5	BDD Example	The pancake Given/When/Then scenario.
BDD p.6	Classical frameworks	Cucumber, JBehave, Concordion, Fitnesse, RobotFramework → "Additional Maintenance Cost".
BDD p.7	Developer-friendly frameworks	Spock, ScalaTest, Jnario, Serenity, JGiven.
BDD p.8	Section divider	"JGiven".
BDD p.9	JGiven intro	Developer-friendly, readable, modular/reusable, reports for experts, open source.
BDD p.10	Scenarios in JGiven	Pancake scenario as a JUnit <code>@Test</code> method (given/when/then/and).
BDD p.11	Stage classes: general	Modularity + reuse; unique to JGiven; one stage per G/W/T.
BDD p.12	State transfer (1)	Diagram: state flows Given→When→Then across coloured stage bands.
BDD p.13	State transfer (2)	<code>@ScenarioState</code> ; values written/read between stages; <code>@Provided</code> / <code>@Expected</code> alternatives.
BDD p.14	Stage example (1)	<code>GivenIngredients</code> stage with <code>@ProvidedScenarioState</code> ingredients list.
BDD p.15	Stage example (2)	<code>WhenCook</code> stage: <code>@Autowired @ScenarioState Cook</code> , <code>@Expected</code> ingredients, <code>@Provided</code> dough/meal; AssertJ sanity checks.
BDD p.16	Stage example (3)	<code>ThenMeal</code> stage: <code>@Expected</code> meal; <code>assertThat(meal).isEqualTo("pancake")</code> .
BDD p.17	Console output	Scenario printed back as Given/And/When/And/Then prose.
BDD p.18	HTML5 app	Browsable "JGiven Report" (53 successful, 0 failed/pending; tags, classes, data tables).

Pages	Deck section	Content
BDD p.19	TNG experience	3 yrs, ≤70 devs, 3000+ scenarios; readability/reuse up, maintenance down, well accepted.
BDD p.20	JGiven summary	Just Java; JUnit/TestNG; Apache 2; Maven/Jenkins plugins; experts can't write scenarios .
BDD p.21	Section divider	"AssertJ".
BDD p.22	Motivation	JUnit underpowered; Fest abandonware; Hamcrest stagnant/ugly.
BDD p.23	Why AssertJ	Maintained; near-superset; well designed; easy to start/enhance/read.
BDD p.24	Basic use	<code>assertThat()</code> factory; <code>AbstractAssert</code> ; fluent API; polymorphism.
BDD p.25	Basic use example	Fluent <code>assertThat(...).contains().startsWith()</code> on String and array; polymorphic.
BDD p.26	Custom condition	Custom conditions for elaborate tests, applied to types with custom assertions.
BDD p.27	Custom condition example	<code>Condition<Integer></code> functor (<code>matches</code>), used with <code>.is()</code> / <code>.isNot()</code> .
BDD p.28	Custom assertion	Subclass <code>AbstractAssert + Assertions</code> → <code>assertThat(student).isInMiddleSchool()</code> .
BDD p.29	AssertJ-Swing	GUI automation: drag 'n drop, component lookup, screenshots on failure, EDT threading checks.
BDD p.30	Getting started	Pointers to AssertJ's "one minute" and "getting started" guides.
BDDLa b p.1	Lab: User Stories → BDD	User-story template; figure mapping 3 stories to GWT scenarios; portfolio: map stories, automate with JGiven + AssertJ (+ AssertJ-Swing for Swing).